

Decoupling Data Layouts from Bounding Volume Hierarchies

CHRISTOPHE GYURGYIK, Stanford University, USA

ALEXANDER J ROOT, Stanford University, USA

FREDRIK KJOLSTAD, Stanford University, USA

Bounding volume hierarchies are ubiquitous acceleration structures in graphics, scientific computing, and data analytics. Their performance depends critically on data layout choices that affect cache utilization, memory bandwidth, and vectorization—increasingly dominant factors in modern computing. Yet, in most programming systems, these layout choices are hopelessly entangled with the traversal logic. This entanglement prevents developers from independently optimizing data layouts and algorithms across different contexts, perpetuating a false dichotomy between performance and portability. We introduce SCION, a domain-specific language and compiler for specifying the data layouts of bounding volume hierarchies independent of tree traversal algorithms. We show that SCION can express a broad spectrum of layout optimizations used in high-performance computing while remaining architecture-agnostic. We demonstrate empirically that Pareto-optimal layouts (along performance and memory footprint axes) vary across algorithms, architectures, and workload characteristics. Through systematic design exploration, we also identify a novel ray tracing layout that combines optimization techniques from prior work, achieving Pareto-optimality across diverse architectures and scenes.

CCS Concepts: • **Software and its engineering** → **Compilers; Domain specific languages**; • **Computing methodologies** → *Ray tracing; Collision detection*.

Additional Key Words and Phrases: acceleration structure, data independence, augmented tree, specialization

ACM Reference Format:

Christophe Gyurgyik, Alexander J Root, and Fredrik Kjolstad. 2026. Decoupling Data Layouts from Bounding Volume Hierarchies. *Proc. ACM Program. Lang.* 10, PLDI, Article 175 (June 2026), 39 pages. <https://doi.org/10.1145/3808253>

1 Introduction

In high-performance graphics, scientific computing, and data analytics, bounding volume hierarchies (BVHs) are essential acceleration structures used for spatial queries such as ray tracing, closest point queries, and collision detection. Extensive research has focused on optimizing the layout of these structures due to their substantial impact on performance [7, 9–11, 16, 18, 19, 23, 25–28, 35, 37, 38, 41, 45, 48, 50, 54, 55, 57, 60, 61, 64, 65, 69, 71, 76, 84, 86, 89, 92, 96, 97, 99–101, 103–106].

However, no single layout is universally optimal. The Pareto frontier of layouts—balancing runtime performance and memory utilization—depends critically on three factors: the algorithm, hardware architecture, and characteristics of the input data. Layouts optimized to maximize performance on CPUs may increase the BVH memory footprint to reduce instruction count or improve cache utilization on latency-bound workloads [10, 101, 105]. On the other hand, memory-efficient layouts are essential for bandwidth-bound systems such as GPUs [41, 54, 63, 82] and memory-constrained platforms such as mobile devices [53, 58, 72, 85], where compact representations directly reduce memory traffic and enable larger scenes to fit in limited memory. Scene properties further

Authors' Contact Information: [Christophe Gyurgyik](#), Stanford University, Stanford, USA, cpg@cs.stanford.edu; [Alexander J Root](#), Stanford University, Stanford, USA, ajroot@cs.stanford.edu; [Fredrik Kjolstad](#), Stanford University, Stanford, USA, kjolstad@cs.stanford.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/6-ART175

<https://doi.org/10.1145/3808253>

complicate the picture, e.g., extremely sparse scenes will benefit from different representations than dense scenes, and the spatial distribution of primitives affects the efficacy of quantization and compression schemes. This produces a substantial exploration space: even a conservative enumeration of k data layouts $\times m$ machines $\times n$ algorithms $\times p$ scenes yields a multifarious set of evaluation contexts, each admitting a potentially different Pareto-optimal solution.

Exploring this design space is complex because general-purpose languages tightly couple data layouts with application logic. Optimized layouts employ diverse techniques: global layout transformations, e.g., struct-of-arrays and hybrid variants [19, 23, 54, 104, 105]; bit field exploitation, e.g., union types [10, 19, 26, 69, 76]; increased branching factors (arity of the tree) [23, 28, 29, 105, 106]; specialized bounding volume representations [45, 46, 55, 105]; implicit indices [7, 19, 25, 86]; and compression [10, 19, 38, 65, 84]. Each of these techniques requires pervasive changes throughout the application code, in how fields are accessed, nodes are referenced, and the tree is traversed.

Orthogonally, programming language research has developed ways to decouple *logical representations* from *physical layouts*. This prior work has focused on fine-grain, per-node layout optimization [6, 15] or coarse-grain composition of data structures hidden behind iterator interfaces [39, 51, 62, 77]. However, state-of-the-art BVH layout optimization requires coordination across both levels of granularity: how individual nodes are represented and how collections of nodes are organized in memory.

We propose a decoupling with control over both. Our key insight is that manual BVH layout optimizations explore different physical realizations of the same underlying logical structure. This enables a clean separation between a tree’s logical specification and its physical representation. This separation preserves the portability and maintainability of traversal algorithms while simultaneously enabling performance engineers to independently tune physical layouts for specific evaluation contexts. Akin to the separation of algorithm and schedule [43, 78], we introduce domain-specific languages to separate algorithm and physical layout. Our primary contributions are the following:

- Two domain-specific languages that decouple a tree’s logical data structure from its physical representation: a *layout* language for specifying physical-to-logical mappings, and a *build* language for expressing the inverse logical-to-physical transformation.
- A compilation strategy for *destructor specialization* that lowers pattern matching onto different physical layouts in accordance with the layout specification.
- A complementary compilation strategy for *constructor specialization* that generates layout-specific constructors from canonical algebraic data types.

We implement these ideas in SCION¹, a system that decouples the specification and traversal logic of tree data structures from their physical representations. This separation enables expressive and portable traversal implementations while supporting systematic exploration of the physical representation design space. Our evaluation demonstrates that layout performance varies significantly across algorithms, architectures, and data characteristics, confirming that design space exploration is essential for Pareto-optimal performance. Through this exploration, we discover a novel layout that composes optimizations from prior work in a previously unexplored way, achieving Pareto optimality across 35 of 42 evaluation contexts. Additionally, we provide evidence that SCION-generated code is competitive with state-of-the-art libraries when using equivalent traversal algorithms, establishing that our abstraction imposes no performance penalty.

Finally, we clarify the scope of this work by identifying what we intentionally exclude. First, we treat tree topology as fixed (*logical tree* in Section 5) and focus solely on layout optimization. While tree quality plays a significant performance role, it is an orthogonal problem with extensive prior

¹The name draws from botanical terminology: *scion* is the grafted portion of a plant chosen to yield particular traits. Analogously, our approach grafts *data layouts* onto acceleration trees to realize targeted performance properties.

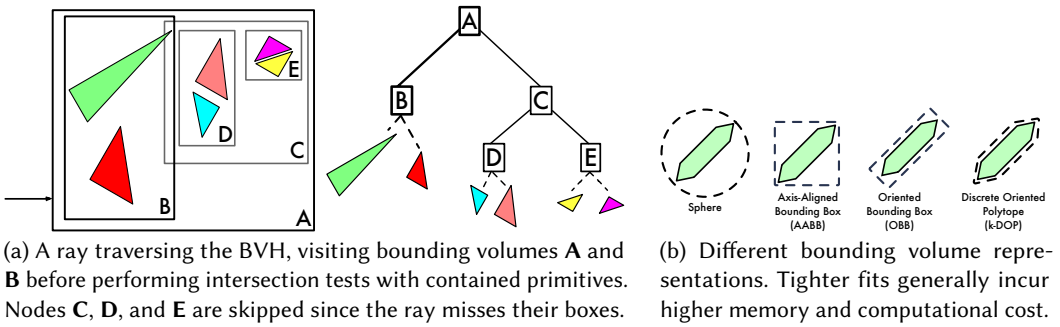


Fig. 1. Visualization of bounding volume hierarchies.

work [4, 8, 31, 47, 56, 67, 75, 102]. SCION explicitly separates logical and physical tree representations to facilitate future integration of topology optimization. Second, we do not explore execution strategies such as vectorization [101, 105], ray reordering [68], and packet tracing [2, 12]. SCION deliberately isolates data layout decisions from scheduling, treating them as separate dimensions of the optimization space. We employ techniques like parallelized outer loops and software-defined stacks for evaluation, but leave a dedicated scheduling language as important future work. While SCION does not perform instruction selection, specifying layouts is a critical first step as automatic vectorization relies on regular layouts [1, 14, 93]. Third, we target read-only acceleration structures, the norm in high-performance systems [10, 37, 45, 54, 69]. Mutability is deferred for future work.

2 Background: Bounding Volume Hierarchies

Bounding volume hierarchies (BVHs) are tree-structured indexes that are widely used in data analytics, computer graphics, and scientific computing to accelerate spatial queries. Each internal node stores a bounding volume that encloses the geometry of its descendants, while each leaf stores one or more geometric primitives. As illustrated in Figure 1a, the root spans the entire scene, forming a spatial decomposition that enables logarithmic-time pruning of the query space [81].

Despite their conceptual simplicity, BVHs exhibit a wide range of performance tradeoffs stemming from their *data representation*. Implementations vary in how bounding volumes, nodes, and primitives are stored, aligned, and traversed. Layout choices affect traversal cost and memory footprint in complex machine-dependent and data-dependent ways, as demonstrated in prior work [38, 45, 105]. This sensitivity makes BVHs an ideal case study for our system: even modest structural changes, e.g., fusing internal and leaf node arrays, can shift the performance frontier. For a detailed introduction to BVHs and surveys of existing optimization techniques, we refer readers to Pharr et al. [76, Ch. 7.3], Ericson [27, Ch. 6], and Meister et al. [69].

3 Overview and Programming Model

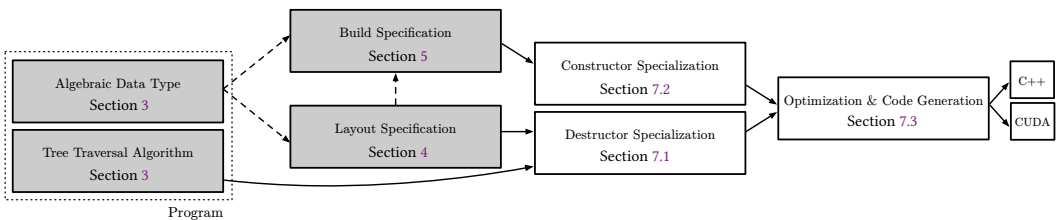


Fig. 2. The SCION system overview. Gray boxes are inputs, solid arrows denote lowering dependencies, and dashed arrows represent use dependencies, e.g., the layout specification is written with respect to the ADT.

Figure 2 illustrates the SCION system overview. Application developers express tree traversal algorithms against algebraic data type (ADT) specifications that define the logical structure of the tree, while performance engineers separately specify how those ADTs are physically realized using the *layout* language (Section 4) and *build* language (Section 5). The layout specification enables *destructor² specialization*: the compiler generates code that extracts an ADT term’s logical values from physical storage (Section 7.1). Conversely, the build specification enables *constructor specialization*: the compiler generates layout-specific constructors that populate physical memory in accordance with the layout mapping (Section 7.2). Lastly, the compiler performs domain-specific optimizations and emits backend code for CPUs or GPUs (Section 7.3).

We note that, while automatic layout inversion is appealing, it is impractical for many important optimizations. Critical optimizations, e.g., quantization for watertight traversal [10, 38, 41, 69], involve non-injective operations that preclude straightforward synthesis (evident in more complex examples provided in Appendix D). Therefore, the build specification must be explicitly stated.

Figure 4 illustrates a closest-hit ray tracing query over a standard binary BVH written in SCION’s tree traversal language (syntax provided in Figure 3). The **BVH** ADT is declared with two variants: **Interior** nodes, which contain a bounding box and child references, and **Leaf** nodes, which contain a bounding box and triangle primitives. For brevity, the syntax on Line 3 states that *each* variant stores a bounding box (bounds). The traversal algorithm on Lines 4–14 matches on these variants, and only recurses on **Interior** nodes or checks for ray-triangle intersection for **Leaf** nodes after verifying the ray intersects the bounding box. We acknowledge that writing efficient tree traversal algorithms is equally critical to overall performance. We leverage BONSAI [80] to automatically generate all traversal algorithms presented in this paper.

Crucially, this algorithm is written with respect to the logical field names, e.g., `left`, and makes no assumptions about how these fields are represented in memory. This separation realizes a form of *data independence* [20] for performance-critical data structures. Logical definitions state *what* the data structure contains and the subsequent operations performed, while separate physical specifications state *how* that data is represented, stored, and accessed.

```

1 type Ray(origin: f32x3, direction: f32x3, tmax: f32 = ∞);
2 type AABB(low: f32x3, high: f32x3); type Triangle(p0: f32x3, p1: f32x3, p2: f32x3);
3 type BVH(bounds : AABB) = Interior(left: BVH, right: BVH) | Leaf(nprims: u16, data: Triangle[nprims]);
4 func closest_hit(ray: Ray, bvh: BVH, best: mut (f32, Triangle)) =
5   match bvh {
6   | Interior(bounds, left, right) ->
7     if intersects(ray, bounds) && (distmin(ray, bounds) < best[0]) {
8       closest_hit(ray, left, best); closest_hit(ray, right, best);
9     }
10  | Leaf(bounds, nprims, data) ->
11    if intersects(ray, bounds) {
12      foreach t in data { if intersects(ray, t) && distmin(ray, t) < best[0] { best = (distmin(ray, t), t); } }
13    }
14  }

```

Fig. 4. A closest-hit ray tracing query algorithm written with respect to the logical BVH specification.

²We use *destructor* in the categorical sense: operations that eliminate ADT values, dual to *constructors* that introduce them.

F : Function	::=	func $x(p^*) \rightarrow T = s$	
p : Parameter	::=	x : mut ² $T (= e)^?$	
s : Stmt	::=	e	
		$x = e$	store
		$s ; s$	sequence
		return $e^?$	return
		match $e \{ A^* \}$	pattern match
		let x : $T = e$	local binding
		if $e_1 \{ s_1 \} (\text{elif } e_2 \{ s_2 \})^* (\text{else } \{ s_3 \})^?$	conditional
		foreach x in $e \{ s \}$	loop
e : Expr	::=	x	variables
		n	literals
		$x(e^*)$	function call
		$T(e^*)$	constructor
		$e_1[e_2]$	index
		$e_1[e_2 : e_3]$	slice
		e as t	type cast
		e to t	bit cast
		$e_1 + e_2 \mid e_1 - e_2 \mid \dots$	operators
A : Arm	::=	$\mid q \rightarrow s$	
q : Pattern	::=	$T_0((x : T)^*)$	variant
		$-$	wildcard
T : Type	::=	t	primitives
		$[T[n]]$	statically-sized fixed array
		$[T[x]]$	dynamically-sized fixed array
		set $[T]$	set
		option $[T]$	optional
		$(T_1, \dots, T_i)$	tuple
		$T \times n$	vector

Fig. 3. Syntax of the traversal language.

```

1 struct LinearBVH {
2   uint32_t P;
3   uint32_t N;
4   Triangle* primitives;
5   LinearBVHNode* nodes;
6 };
7
8 struct alignas(32) LinearBVHNode {
9   AABB bounds;
10  union {
11    uint32_t p_o; // Leaf (primitive offset)
12    uint32_t c_o; // Interior (2nd child offset)
13  };
14  uint16_t nprims; // 0 -> Interior, > 0 -> Leaf
15 };

```

(a) Original PBRTv4 layout in C++.

```

1 layout LinearBVH(I: u32) {
2   P: u32; N: u32; primitives: Triangle[P];
3   group nodes[size=N, align=32] by I {
4     bounds: AABB;
5     split nprims {
6       > 0 -> Leaf {
7         p_o : u32; data = primitives[p_o : p_o + nprims];
8       };
9       0 -> Interior {
10        c_o : u32; left = I + 1; right = I + c_o;
11      };
12    };
13    nprims: u16;
14  };
15 };

```

(b) PBRTv4 layout in SCION.

Fig. 5. Comparison of PBRTv4 BVH layouts for the logical BVH on Line 3 of Figure 4.

4 Physical Layout Language

Separating layout from algorithm requires addressing how representation choices affect both storage and traversal. An ADT term might be referenced via a stored 32-bit array index or by implicitly computing its location from the parent's position. Each choice changes not only the physical footprint but also the operations needed to traverse the structure, and thus requires specifying how to locate ADT terms in memory and how to interpret their physical representation once located. To address this, the layout language expresses two complementary pieces of information: (1) the concrete type that encodes an ADT *reference*—the representation used to uniquely identify a term in memory—and (2) the interpretation of that representation, i.e., how to determine which variant it denotes and how to derive its fields. We begin by examining a layout from a standard rendering textbook [76], showing how typical optimizations alter both representation and interpretation in intertwined ways, and then define layout primitives that generalize these transformations.

4.1 PBRTv4 Layout Example

The PBRTv4 rendering system [76] performs ray tracing as illustrated in Figure 4, but with the C++ data structures shown in Figure 5a for its BVH data structure. Its design incorporates several optimizations that deviate from a standard pointer-based encoding of the BVH ADT:

- (1) Nodes reside in a contiguous array; children (left, right) are 32-bit unsigned indices into it.
- (2) The left child is stored directly after the parent, eliminating an explicit reference.
- (3) **Interior** data and **Leaf** data are unified, discriminated by the `nprims` field.
- (4) Triangles reside in a separate array; **Leaf** nodes store an offset into it and a primitive count.

To achieve the goal of combining clarity of ADTs with specialized layout performance, SCION provides a separate declarative layout language. Figure 5b presents the PBRTv4 layout written in SCION. Line 1 specifies that an ADT term can be referenced by `I` of type `u32`. Because multiple arrays may be indexed by the same reference, e.g., separate arrays for different variants that share a common index space, the reference type is declared in the layout signature rather than within individual **group** declarations. This enables `I` to serve as a reference across one or more groups. Line 2 declares global fields: the number of primitives, the number of nodes, and an array of triangles. Lines 3–14 declare an array of nodes. The **group** declaration establishes the size of the array named `nodes` and the alignment of each element, as well as how the array is indexed (via `I`). Lines 4 and 13 specify fields that each node stores: the bounding box and number of primitives respectively. Lines 5–12 define a safe tagged union discriminated by the number of primitives. This is justified by the BVH invariant that only leaves contain primitives.

4.2 Layout Primitives

SCION's layout language provides a small set of composable primitives for describing the exact layout of a tree in memory (abstract syntax provided in Figure 6). A layout specification defines:

- (1) A *reference* type: the concrete type that represents a logical reference to an ADT term. This enables coarse-grain optimization, such as storing ADT terms in struct-of-arrays format.
- (2) How that reference type should be interpreted: which ADT variant it represents and how the fields of that variant should be loaded. This enables fine-grain optimization, such as bit-stealing (using bits that are otherwise reserved for padding or pointer alignment).

A layout in SCION begins by declaring the reference type: Line 1 of Figure 5b does this via `layout LinearBVH(I: u32)`, which specifies that each term of the ADT is uniquely identified by a value of type `u32`. The parameter `I` serves as a bound identifier for the reference in the rest of the layout specification. Reference types can be raw pointers, indices into arrays, composite structures with multiple indices, or richer encodings described in Section 4.2.3. The primitives specifying the interpretation of a reference type operate at three conceptual levels:

- (1) *Local*: how bits correspond to fields,
- (2) *Structural*: how values *compose* into collections, and
- (3) *Relational*: how dependencies *span* collections.

4.2.1 Local. SCION supports two notions of *fields*, a *stored* field and a *derived* field. *Stored fields* assign an identifier to a region of bits in memory. For example, `x: f16` defines a field named `x` that occupies 16 bits and has type `f16`. Conceptually, a field behaves like an l-value [88]: it refers to a location in memory that can be read or written. Accessing a field performs a load from memory. In PBRT's BVH in Figure 5b, bounds on Line 4 is an example of a stored field. The offsets `p_o` and `c_o` are also stored fields, but they do not directly correspond to *logical* fields of the ADT. They are instead used when deriving other logical fields of the ADT.

Derived fields bind a name to an expression, i.e. an r-value³, as in `x = 42`. Some fields might be pure functions of other fields, i.e., chosen not to be physically stored in the layout. Derived fields are evaluated lazily, and at most once per destruction of the associated field; this may be elided by compiler optimizations. `left` and `right` on Line 10 of Figure 5b are examples of derived fields: they are computed from the reference and a stored field, `c_o`. Together, stored and derived fields span the entire spectrum of compute–memory tradeoffs, from purely stored to purely derived representations. The remaining primitives serve only to provide useful abstractions that express the layout of *collections* of terms.

4.2.2 Structural. Structural primitives specify how stored and derived fields are organized into aggregates and variants. They provide the operators that enable the composition of fields into variants (via `split`) and control the layout of *collections* of fields or variants (via `group`).

Groups define an ordered collection of sub-layouts and provide the mechanism for expressing global layout transformations such as tiling data along the array-of-structs (AoS) to struct-of-arrays (SoA) spectrum, including hybrid variants used in vectorized code [10, 105]. Every group is indexed by a value that determines how its elements are addressed in memory. Typically, the value is

ℓ : Layout	::=	<code>layout</code> $x(p^*) \{ M \}$	
p : Parameter	::=	$x : T (= e)^?$	
M : Member	::=		
		$x : T$	field
		n	padding
		$x = e$	derive
		<code>group</code> $id [K^*] (by\ x)^? \{ M \}$	group
		<code>indirect group</code> $id \{ M \}$	indirect group
		<code>split</code> $e \{ A^+ \}$	split
		<code>let</code> $x : T = e$	local binding
		$M; M$	sequence
A : Arm	::=	$p \rightarrow M$	
		$p \rightarrow T \text{ from } x[e]$	foreign key
K : Attribute	::=	n	literals
		$size = n$	cardinality
		$align = n$	alignment
p : Pattern	::=	n	literals
		$ > < \geq \leq \sim n$	compare
		$-$	wildcard
e : Expr	::=	<code>parent.x</code>	tree dependency
		$\cup \{ e \in \text{Tree Traversal Language} \}$	

Fig. 6. Syntax of the layout language.

³Also referred to as a *method call* on a class in object-oriented programming or a *computed column* in databases.

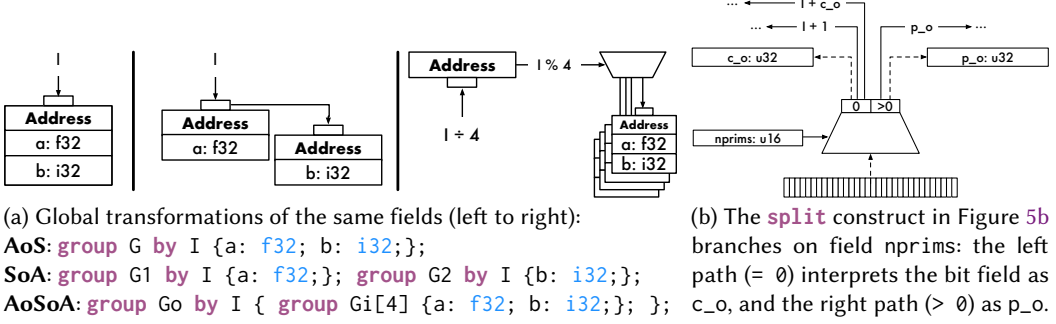


Fig. 7. Example sub-layout diagrams. Solid arrows indicate the flow of data (either a derivation or loaded field), and dashed arrows represent interpretation of previously unnamed bits by the **split** construct.

an integral-type index, indicating a contiguous array, but could alternatively be a pointer type, stipulating a logical grouping of structures that are stored arbitrarily in memory.

Groups come in two forms: direct and indirect. Direct groups are indexed by a component of the reference type and correspond to primary storage. In the PBRTv4 layout, the group starting on Line 3 of Figure 5b is a direct group: it is indexed by the reference value `I`. Indirect groups, labeled via **indirect**, represent auxiliary storage (akin to *foreign tables* in relational databases) and are accessed from direct groups using **from** (a relational primitive, discussed in Section 4.2.3). As demonstrated in Section 8.2, indirect groups are particularly effective for heterogeneous data, where different ADT variants have disparate storage requirements.

Composing groups facilitates specifying complex data layouts. An AoS layout uses a single group with multiple fields, while a SoA layout uses adjacent single-field groups that share a common index; these are illustrated in Figure 7a. Hybrid layouts, e.g., AoSoA, are specified by nesting groups, where the inner group defines a tile size. We also provide a primitive for declaring SoA layouts within a single group when each group shares the same index. The separator `---` splits a group into multiple adjoining groups while preserving lexical scope so derivations can refer to one another.

Splits provide a mechanism for safe tagged unions, as in the RIBBIT compiler [6]. The **split** e construct expresses conditional branching and bit interpretation based on the value of the expression `e`. We implement a subset of the split construct provided in RIBBIT, tailored for the tagged unions found in BVH layout optimization. Each arm of the split defines a unary condition, where simple constants like `0` are implicitly treated as equality tests, and other comparisons must be stated explicitly, e.g., `< 42` or `≥ 0`. The wildcard symbol `_` serves as a catch-all for any unmatched cases. The right-hand side of a split arm specifies the corresponding sub-layout and its variant type. For example, Lines 5–8 of Figure 5b specify that when `nprims` is greater than `0`, the sub-layout storing the `p_o` field and deriving the data array should be interpreted as a **Leaf** variant.

The primitives so far describe how individual ADT terms and the entire collection are organized in memory. We further need to describe how groups are connected, i.e., how local primitives of an indirect group are accessed, and how recursive datatypes are instantiated.

4.2.3 Relational. SCION’s relational primitives define how data in one part of the layout refers to or depends on data in another. These operators generalize pointer dereferences and *foreign-key* lookups, enabling layouts to specify both explicit references between disjoint groups and implicit dependencies along a tree’s hierarchy. SCION supports two core relational primitives: a **from** operator that acts as a load from an **indirect** group via a foreign key and the construction of complex primary keys, i.e., the reference type, with *tree-carried dependencies*.

An **indirect** group is not directly indexed by a component of the reference type. Consequently, fields in indirect groups have different scoping rules than those in direct groups, as we discuss in Section 6. To access the fields represented in an **indirect** group, the **from** operator is used in conjunction with a key or range of keys. A concrete example of this is provided in Section 8.2.

While **from** handles relationships between disjoint groups, recursive references capture relationships *within* the hierarchy. These references must respect the layout’s declared reference type, which determines how a node identifies and communicates with its children. Similarly, any field that is originally a recursive datatype in the ADT language must be loaded or derived with a type that matches the specified reference type of the tree. While the reference type is typically an index, a set of indices, e.g., nested direct groups, or a pointer type, SCION additionally supports tree-carried dependencies, also referred to as hierarchical encoding [64, 84].

Tree-carried dependencies enable terms to share information with recursive ADT fields, reducing per-term storage by passing shared data into the reference when destructuring. Since a reference becomes incomplete without parent context, the parent information is incorporated into the reference itself and thus the **layout** signature. The **parent** prefix is used to specify these dependencies. For example, **parent**.*x* is the parent’s value of *x*. As demonstrated in Section 8.3, the layout signature must indicate that such dependencies are components of the reference type.

4.3 Layout Diagrams

To aid understanding of layout specifications, we introduce *layout diagrams* that visualize how field accesses propagate through layout language constructs. These diagrams trace the path from the initial reference (illustrated as a shaded box) through **group** constructs, **split** primitives, and **indirect** lookups, terminating at either stored fields (loads from memory) or derived fields (computed values). Solid arrows represent data flow with associated semantics: an unlabeled arrow with a field origin denotes a direct memory load, while a labeled arrow with an expression, e.g., $I + c_o$ in Figure 8, denotes a derived field computed from previously accessed values. Dashed arrows represent type reinterpretation of unnamed bit fields (visualized as individual bits) performed by the **split** construct. At any point in the diagram, all previously encountered named fields are accessible and bound to their corresponding memory addresses or computed values. The diagram thus captures both the data flow through structural and relational primitives, and scoping of local primitives during traversal, further explained in Section 6. For brevity, we elide derivation expressions for fields whose values are directly copied from memory.

Importantly, layout transformations correspond directly to structural manipulations of these diagrams: global layout transformations like AoS-to-SoA duplicate group structure (Figure 7a), **split** constructs introduce conditional branching or reinterpretation of bits (Figure 7b), and relational lookups, i.e., **from**, create inter-group edges (Section 8.2). We now describe how these primitives compose *correctly*. We next discuss how the build language specifies the *constructor* (logical-to-physical mapping).

5 Build Language

A goal of SCION is to provide a declarative specification for transforming logical trees into their physical representations while maintaining clarity of the algebraic structure. Recall that the *logical* tree is a straightforward realization of the algebraic data type, where each constructor maps

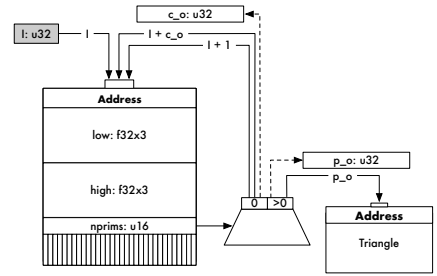


Fig. 8. Layout diagram for PBRTv4.

to a variant and fields are stored with their declared types. The *physical* tree is the optimized representation obtained by transforming the logical tree according to the build⁴ specification. The layout language specifies the memory layout, while the build language specifies the transformation from logical tree to physical tree.

Together, layout and build languages express a bidirectional mapping between logical and physical representations. The *layout* defines the destructor: how to extract logical fields from a physical tree (physical $\rightarrow$ logical). The *build* defines the constructor: how to populate physical storage from a logical term (logical $\rightarrow$ physical).

These specifications must be consistent with each other. A field that appears in a layout's **split** branch of **group** must be populated by a corresponding **build** statement. Conversely, every **build** statement must target a field that exists in the layout. However, the two specifications serve complementary roles and need not be symmetric, e.g., the layout may define additional derivations, while the build only populates stored fields.

5.1 Two-Phase Construction

Prior work has used two strategies for construction of acceleration structures. *Single-phase* construction builds the specialized physical layout directly during tree construction, avoiding intermediate representations. This was the approach taken by Burtscher and Pingali [13] for a class of space-partitioning trees, and is attractive when construction latency is critical, e.g., dynamic scenes. *Two-phase* construction first builds a canonical tree (with an unoptimized layout) and then packs it into the target physical layout [5, 76]. This approach incurs additional latency and memory overhead from intermediate naïve layout construction, but cleanly separates tree topology optimization from physical layout optimization, thus simplifying end-to-end construction.

We adopt the two-phase approach: SCION assumes the tree topology is fixed before layout specialization begins. This assumption is natural for static scenes, where construction costs are amortized over many traversal queries, and it additionally enables the build language to treat the logical tree as immutable input.

5.2 PBRTv4 Build Example

We illustrate the build language via the PBRTv4 example before defining its primitives. Figure 9 presents the build specifications for the LinearBVH layout. Line 1 declares that the tree is *laid out* in preorder traversal, i.e., the parent node is built before its children are recursively visited. Line 2 declares the **Interior** variant signature, containing the original fields from the logical ADT. Line 3 copies a field from the logical tree, i.e., **build** bounds, and stores the discriminant value (**build** nprims = 0). Line 4 builds the left and right subtrees, saving the reference of the right child. Line 5 computes the relative offset to the right child (**build** c_o = R - **this**), leveraging the preorder layout to elide storing the left child reference. Lastly, it returns the current node's reference. The **Leaf** variant follows a similar pattern: copying stored fields (Line 8), appending primitive data to the indirect array (Line 9), and returning the node reference (Line 10). Together, these specifications completely determine the logical-to-physical tree constructor, using the layout mapping to determine *where* these values live in memory. Next, we define the semantics of primitives in the build language.

```

1 build LinearBVH[order=pre] {
2   build Interior(bounds : AABB, left: BVH, right: BVH) {
3     build bounds; build nprims = 0;
4     build left; let R: u32 = build right;
5     build c_o = R - this; return this;
6   };
7   build Leaf(bounds : AABB, nprims: u16, data: Triangle[nprims]) {
8     build bounds; build nprims;
9     build p_o = append(data, nprims);
10    return this;
11  };
12 };

```

Fig. 9. Build specification for the PBRTv4 layout.

⁴In this work, *build* and *construction* have distinct definitions. Build refers to the transformation from *logical* tree to *physical* tree (in scope), and construction refers to the assembly of the logical tree (out of scope).

5.3 Build Primitives

The build language provides a small set of primitives augmented by the tree traversal language for describing how to construct physical trees from logical trees. Each build specification defines, for each variant of the algebraic data type, the sequence of operations required to derive that variant in the physical layout. The constructs operate at two conceptual levels: *local* primitives and *structural* primitives. The abstract syntax is provided in Figure 10.

5.3.1 Local. Local primitives describe how stored (physical) fields are populated for each variant type. *Build* statements populate physical fields from logical values. The statement **build** x copies the field x directly from the logical representation to the corresponding physical location in the layout specification. For example, in Figure 9, **build** low copies the low field from the logical tree to its layout-specified memory location (handled automatically by the compiler). Build statements can also compute values: in **build** $x = e$, e is evaluated in the context of the current logical node, and the result is stored in x 's physical location. For example, the **Interior** node declares **build** $nprims = 0$ to store 0 in the layout's $nprims$ field. This primitive enables layout-specific encodings absent from the logical representation, such as discriminant tags or auxiliary metadata. We also provide syntactic sugar for appending to an **indirect** group: $append(x, n)$ appends n elements of x and returns the starting position, with the location automatically inferred by the layout mapping.

β : Build ::= build $x[K] \{ V^+ \}$	
K : Attribute ::=	
$order = \{pre, post\}$	ordering
V : Variant ::= build $T_0((x: T)^*) \{ R^? ; s \}$	
R : Root ::= build root $\{ s \}$	
s : Stmt ::= build $x (= e)^?$	build field
return e	return reference
let $x: T = e$	local binding
$s ; s$	sequence
e : Expr ::= this	current reference
append (x, e)	append to array
$\cup \{ e \in \text{Tree Traversal Language} \}$	

Fig. 10. Build language syntax.

5.3.2 Structural. Structural primitives control the global organization of terms in memory and the construction of references between terms. For **build** statements of fields with recursive algebraic data types, the fields are constructed in a recursive, depth-first manner. For example, **build** $left$ fully materializes the left child and all its descendants before any subsequent statements execute. The order attribute controls when the current term is materialized in memory relative to its children: preorder ($order=pre$) emits the parent node before recursively visiting children, while postorder ($order=post$) emits the parent node after all descendants have been visited. Critically, order affects only the memory layout. It does not constrain dependency resolution, e.g., the parent node may compute fields that depend on values from its children in preorder layouts, provided those values have already been constructed. This is demonstrated concretely on Line 5 in Figure 9, where the current node's field c_o depends on the reference to the right child R .

This is a special identifier that refers to the unique reference of the current term, e.g., an index into a contiguous array. Uses of **this** enable computing relative offsets, e.g., in Figure 9, we compute the right child's offset by subtracting its index from the currently-visited node's index. We can also modify the reference type to hold additional information. For example, a pointer type may only use a 48 bits within a 64-bit field, leaving 12 bits that can be used to store auxiliary information, e.g., **this**[48:63] = metadata.

Return statements produce the reference to the constructed term. The expression e in **return** e must evaluate to a value with the reference type declared in the layout specification. When building a field with a recursive ADT, this value may be used by the parent to realize inter-term dependencies.

Root specifications handle special initializations required at the root of the tree. Some layouts require computing base-case values for inductively defined tree-carried dependencies or initializing global parameters. The **root** block executes before any variant-specific build procedures and can populate these global values. For instance, a quantization scheme might compute the world bounding

box and store it as a global parameter that all nodes use during quantization. Such an example is provided in Appendix D.3.

6 Well-Formedness

A key concern for BVH developers adopting SCION is what correctness guarantees the compiler provides. Specifically, can every logical field be unambiguously accessed in the physical layout, and does encoding a tree and decoding it back preserve semantics? We address this through a combination of static analyses that cover the full language, and proof sketches for the correctness of *derivation-free* specifications. This subset includes non-derived **field** and structural primitives (**group**, **split**), but excludes other constructs (derived **field** and relational primitives) as these may introduce arbitrary computation. Verifying round-trip soundness of derived specifications would require proving functional equivalence of two programs, which is (in general) undecidable or at least requires separate proof obligations [79]. We thus trust that programmers of derived specifications implement correct code.

6.1 Static Analysis

The tree traversal language uses Hindley-Milner typing [36]. For destructor specialization (Section 7.1) to be well-defined, field resolution must produce an unambiguous memory access or derivation for each logical field reference in the traversal code. The compiler statically verifies this by checking that every layout admits a unique physical path to every field of every variant; the algorithm is provided in Appendix A. We also provide additional static checks to eliminate certain classes of bugs. For example, the **split** construct must cover the full domain of its discriminant, the dependency graph built by derived fields must be acyclic, and fields in the physical layout must preserve the typing of the logical specification.

6.2 Formal Guarantees

In Appendix B, we state the well-formedness conditions for the derivation-free subset of SCION. This necessarily deviates from any guarantees about layouts with arbitrary compute possible in the complete language. Informally, we show that if the layout and build are well-formed, then for every variant V and every logical field $(f : T) \in V$, there exists exactly one physical path to it. Consequently, in the derivation-free subset, encoding a tree (via the *build* language) and decoding the tree (via the *layout* language) will always result in the same tree.

The derivation-free subset captures the structural core of SCION but excludes features essential for state-of-the-art BVH optimizations. Derived fields enable bounding-volume quantization and hierarchical compression; tree-carried dependencies permit compression of information shared inductively; and indirect groups support multi-buffer layouts. Extending the round-trip proof sketch to cover them requires substantial proof obligations on semantic correctness, e.g., that quantization followed by dequantization conservatively encloses the original bounding volume. Formalizing these obligations, or intelligently restricting the expressiveness of SCION to guarantee semantic preservation by construction, is important future work.

7 Compilation

The SCION compiler transforms programs into machine code through a set of stages. Layout and build validation ensure well-formedness: unambiguous field paths, complete coverage, and type consistency. These constraints are not merely safety checks. They directly simplify the lowering process by guaranteeing that every field access has a unique resolution path and that every ADT term has a corresponding physical representation. Layout specialization translates declarative specifications into explicit memory address computations, relying on unambiguous field paths

to perform straightforward syntax-directed expansion. Build lowering emits constructor code by matching layout fields against algebraic data type fields, automatically generating the logical-to-physical tree conversion. We then run compiler optimization passes, and finally, the backend emits either CUDA or C++ for existing compiler toolchains.

7.1 Destructor Specialization

Destructor specialization is the process of mapping logical field access in the ADT (destruction) to physical memory, as specified by the layout (specialization). This is conducted through a field resolution process: given a destructed field from the ADT, the compiler determines the corresponding memory access or derivation in the layout specification. Algorithm 1 presents a simplified version of this process. The algorithm operates as a syntax-directed macro expansion: it traverses the layout specification to find the declaration of the requested field, then recursively expands that declaration into concrete memory operations. This approach exploits the compositionality of layout transformations—a field access through multiple primitives expands by recursively applying each transformation’s interpretation rules, yielding a *path*. For example, accessing the bounds field in Figure 5b yields the path ($\mathcal{B}$, nodes[I], bounds): start at the base structure $\mathcal{B}$ (a compiler-generated name), index into the direct group nodes via reference I, and load field bounds. Figure 11 provides the complete destructor specialization of closest-hit ray tracing.

The full implementation of Algorithm 1 handles additional complexities, e.g., scope management for nested contexts. Despite these simplifications, the algorithm embodies the essential mechanism: systematic traversal of the layout to resolve concretized field access patterns that respect all layout transformations. Field resolution is linear in the number of layout members, resulting in quadratic complexity for concretizing an entire tree traversal. This remains practical as layouts are generally compact (tens of members), and the compiler amortizes this cost by caching resolved fields.

Critically, the well-formedness constraints discussed in Section 6 ensure this traversal is always well-defined. Field coverage (and uniqueness) guarantees every queried field has an unambiguous representation, acyclic dependencies ensure derivations can be evaluated in order, and switch exhaustiveness ensures variant disambiguation always succeeds. Consequently, this enables layout-polymorphic queries: the same field access in the traversal algorithm automatically compiles to different concretized data representations depending on the layout specification.

Algorithm 1 Field Resolution

```

1:  $P$ , current memory access path
2:  $M$ , layout member to traverse
3:  $F$ , target field to resolve
4:  $T_D$ , current variant type for SPLIT disambiguation
5:  $I$ , map of indirect group identifier to (layout, path) pairs
6:  $S$ , map for caching derivation paths
7: function CONCRETIZE( $P, M, F, T_D, I, S$ )
8:   match  $M$  with
9:   | FIELD( $id$ )  $\Rightarrow$ 
10:    if  $id = F$  then return ( $P, id$ )
11:    else cache ( $id, (P, id)$ ) in  $S$  for DERIVE
12:    | GROUP( $T_G, body, index$ )  $\Rightarrow$ 
13:      match  $T_G$  with
14:      | DIRECT  $\Rightarrow$ 
15:        if  $index$  is ptr then
16:          return CONCRETIZE( $*P, body, F, T_D, I, S$ )
17:        else
18:          return CONCRETIZE( $P[index], body, F, T_D, I, S$ )
19:      | INDIRECT( $name$ )  $\Rightarrow$ 
20:        insert ( $name, (body, P)$ ) into  $I$ ; return  $\perp$ 
21:      | FROM( $name, key$ )  $\Rightarrow$ 
22:        let ( $body, P_i$ ) :=  $I[name]$  in  $\triangleright$  Fail if  $name \notin I$ 
23:        return CONCRETIZE( $P_i[key], body, F, T_D, I, S$ )
24:      | SPLIT( $discriminant, arms$ )  $\Rightarrow$ 
25:        for each  $Arm(C, T_a, body) \in arms$  do
26:          if  $T_a = T_D$  then
27:            return CONCRETIZE( $P, body, F, T_D, I, S$ )
28:      | DERIVE( $id, E$ )  $\Rightarrow$ 
29:         $V \leftarrow \text{EVALUATE}(E, S)$ 
30:        cache ( $id, V$ ) in  $S$ 
31:        if  $id = F$  then return  $V$ 
32:      | SEQUENCE( $layouts$ )  $\Rightarrow$ 
33:        for each  $L \in layouts$  do
34:          let  $r := \text{CONCRETIZE}(P, L, F, T_D, I, S)$  in
35:          if  $r \neq \perp$  then return  $r$ 
36:   return  $\perp$   $\triangleright$  field not found
37: end function

```

```

1 func closest_hit(
2   ray: Ray, I: u32, PT: LinearBVH, best: mut (f32, Triangle)) =
3   if !(PT.nodes[I].nprims > 0) {
4     if intersects(ray, PT.nodes[I].bounds) &&
5       distmin(ray, PT.nodes[I].bounds) < best[0] {
6       closest_hit(ray, I + 1, best);
7       closest_hit(ray, I + PT.nodes[I].c_o, best);
8     }
9   } else {
10    if intersects(ray, PT.nodes[I].bounds) {
11      foreach t in PT.primitives[
12        PT.nodes[I].p_o : PT.nodes[I].p_o + PT.nodes[I].nprims {
13        if intersects(ray, t) && distmin(ray, t) < best[0] {
14          best = (distmin(ray, t), t);
15        }
16      }
17    }
18  }

```

Fig. 11. Destructor code generation for PBRTv4 layout.

7.2 Constructor Specialization

Constructor specialization generates code that transforms logical ADT terms into their physical representation according to the build specification. Lowering proceeds in linear time, through two phases. First, the compiler analyzes layout specifications to compute the size of each contiguous buffer, ensuring that each buffer requires only a single allocation. Second, the compiler generates code to recursively traverse the logical tree, applying variant-specific build operations at each node: **build** statements handle field values and process children, and **return** statements produce specialized references. Buffer offsets are tracked automatically to ensure correct placement. Each logical node thus produces exactly one specialized node with all physical transformations applied. The complete C++ lowering for Closest Hit Ray Tracing with PBRTv4 layout appears in Appendix F.

7.3 Optimization & Backend Code Generation

A key advantage of DSL-level optimization is the ability to exploit semantic guarantees unavailable to general-purpose language compilers. Consider the field resolution algorithm, which naïvely stamps in fields during tree traversal even when a path was already accessed, resulting in redundant memory operations. Eliminating these redundancies requires sophisticated data flow and memory analysis, which is prohibitively difficult in general-purpose languages. Our tree traversal DSL simplifies this analysis by guaranteeing BVH structures remain immutable during traversal. This immutability guarantee enables aggressive elimination of redundant control flow, and, in conjunction with a simpler intermediate representation, enables straightforward reuse of common subexpressions across control flow boundaries. These optimizations alone yield approximately 1.2–1.4× speedups compared to SCION when DSL-specific compiler optimizations are disabled.

The compiler targets both C++ and CUDA, performing several lowering passes to bridge the gap between the high-level representation and backend languages. The lowering process is relatively straightforward since our tree traversal language is already similar to C. For the CUDA backend, we also perform necessary memory transfers between host and device. Where possible, the compiler maps arithmetic operations to architecture-specific intrinsics, e.g., on CUDA, this includes leveraging built-in math intrinsics for bounding volume quantization following Howard et al. [37].

8 Case Studies

We demonstrate SCION’s expressiveness through three case studies that span the design space of data layout optimizations for BVHs. The first case study, Discrete Oriented Polytopes [45], shows how memory access pattern transformations improve cache utilization by rearranging data. The second, Strand-Based Geometry [105], illustrates heterogeneous representation by combining multiple bounding volume types within the same ADT. The third, Tree-Carried Dependencies [7, 26], demonstrates hierarchical compression, where child nodes inherit and refine parent state rather than storing redundant information. Each example highlights how SCION’s compositional primitives enable concise specification of real data representation optimizations.

8.1 Improving Locality for Discrete Oriented Polytopes

The choice of bounding volume introduces another axis of physical layout decisions, as seen in prior work for spheres [42, 49, 73], axis-aligned bounding boxes (AABBs) [19, 21, 38, 92], oriented bounding boxes (OBBs) [24, 30, 105], k-ary discrete oriented polytopes (k-DOPs) [22, 45, 52], and hybrid variants [46, 55]. Káčerik and Bittner [45] demonstrate that separating 14-plane discrete oriented polytope (DOP-14) bounding volumes into two arrays improves traversal performance by enabling independent memory access patterns; this is illustrated in Figure 12. Their work introduces a novel culling strategy where they first check the AABB planes of the bounding volume (Line 4),

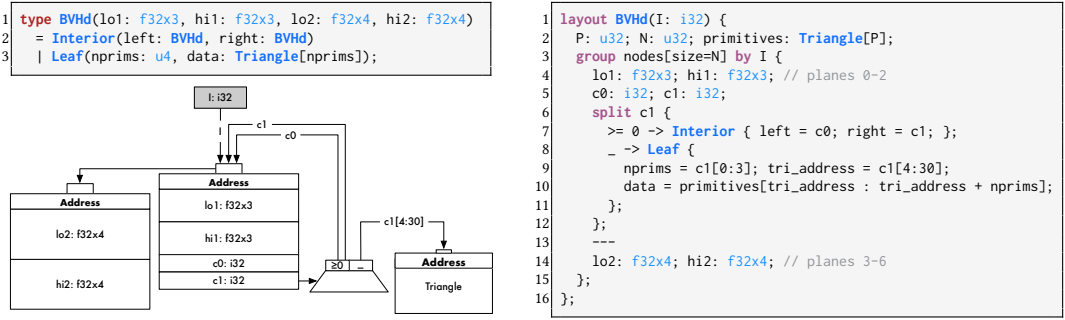


Fig. 12. BVH with DOP-14 bounding volumes [45] specified in Scion.

and only test planes 3–6 (Line 14) if warranted [45, Sec 4.4]. The authors implemented separate traversal algorithms: one for the AoS layout that loads and checks all seven planes at once, and another for the SoA layout that processes separated planes. Our layout language can express the latter transformation by enclosing these fields in a new group (Line 13). Fields `lo1`, `hi1`, `c0`, and `c1` correspond to these AABB planes occupying the first array, and are accessed immediately during traversal. Fields `lo2` and `hi2` reside in a separate array, and are accessed only when the finer-grain intersection test is needed.

This case study demonstrates Scion’s ability to express seemingly complex transformations in a concise manner. Additional changes, e.g., laying out planes in AoSoA for vectorization, can also be readily explored without requiring the traversal to be rewritten. Scion cleanly separates the logical bounding volume specification (which planes comprise a DOP-14) from the physical organization (which planes are co-located in memory). This separation facilitates exploration of alternative bounding volume representations that yield significant performance implications.

8.2 Bit-stealing for Strand-Based Geometry Rendering

Strand-based geometry, e.g., hair and fur, presents unique challenges for ray tracing due to high primitive counts and poor spatial coherence. We demonstrate Scion’s expressiveness through an implementation of the mixed bounding volume strategy from Woop et al. [105], which employs AABBs near the tree root for fast traversal, then transitions to OBBs at deeper levels for tighter

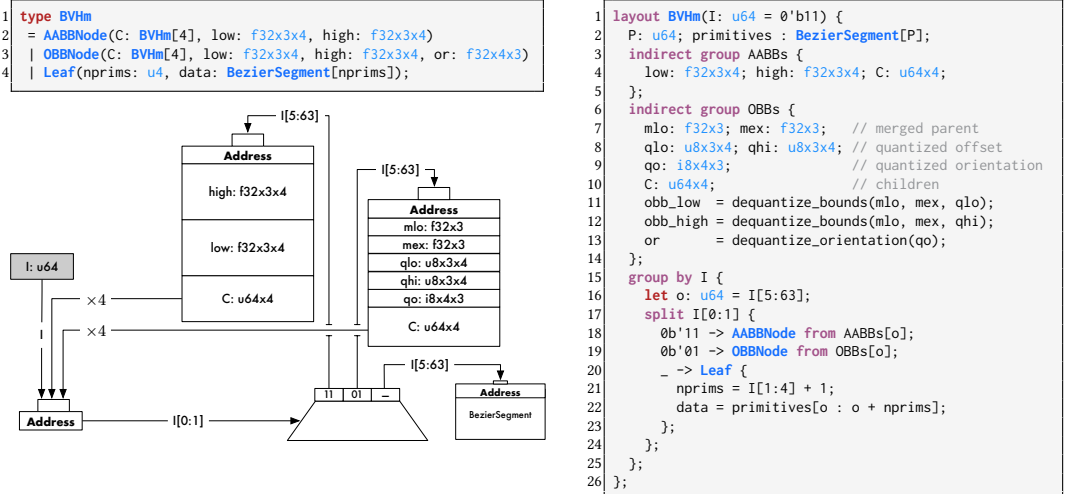


Fig. 13. BVH for strand-based geometry with heterogeneous volumes and quantized OBBs specified in Scion.

volume fits. This heterogeneous representation combines sum types for variant discrimination, a quantized bounding volume encoding, and bit-field exploitation for compact node encoding.

Figure 13 presents Woop’s layout specification in SCION. Lines 16–21 illustrate bit-stealing on index I to encode the node type in bits 0–1, the primitive count for leaves in bits 1–4 (unused in interior nodes), and the offset in bits 5–63 (primitive offset for leaves or node offset for the two different interior variant types). AABB nodes store uncompressed floating-point bounds (Lines 3–5). OBB nodes (Lines 6–14) apply a quantization scheme where a merged-parent bounding volume provides the quantization frame, and per-child bounds encode 8-bit offsets and a shared quantized orientation matrix amortized across all four children as described in Woop et al. [105, Sec 3.5]. For brevity, we do not show the definitions of the OBB dequantization functions used in Lines 11–13.

This case study illustrates two aspects of SCION’s design philosophy. First, the algebraic data type cleanly separates the logical tree structure (three node variants with different fields) from the physical layout (quantization, bit packing, and indirection). Second, the compositional specification enables straightforward exploration of alternative representations: we can easily evaluate applying the AABB compression from Benthin et al. [10, Sec 4.1], or reorganize memory by storing bounding volumes in a shared buffer to improve spatial locality during the AABB-to-OBB transition. These variations require modifying only the layout specification and build language, with the low-level traversal code automatically regenerated by the compiler.

8.3 Reducing Storage Through Tree-Carried Dependencies

Many BVH compression techniques exploit dependence (or hierarchical) relationships where child node representations depend on parent node state [84, 106]. The canonical example is the AABB shared slab optimization [7, 26] expressed in Figure 14. When a node splits along a single axis, the bounding planes orthogonal to that axis remain unchanged for both children. Rather than storing all six bounding planes per child, each interior node stores only the split axis (Line 10) and two split plane positions (Line 8). Children nodes inherit the four unchanged planes by carrying parent bounds through the traversal (Lines 15–17). This optimization reduces redundant storage by having children inherit invariant properties from ancestors rather than storing them explicitly.

This example demonstrates SCION’s support for more complex reference types and thus stateful traversal through its parent mechanism. The logical specification, a binary tree of axis-aligned

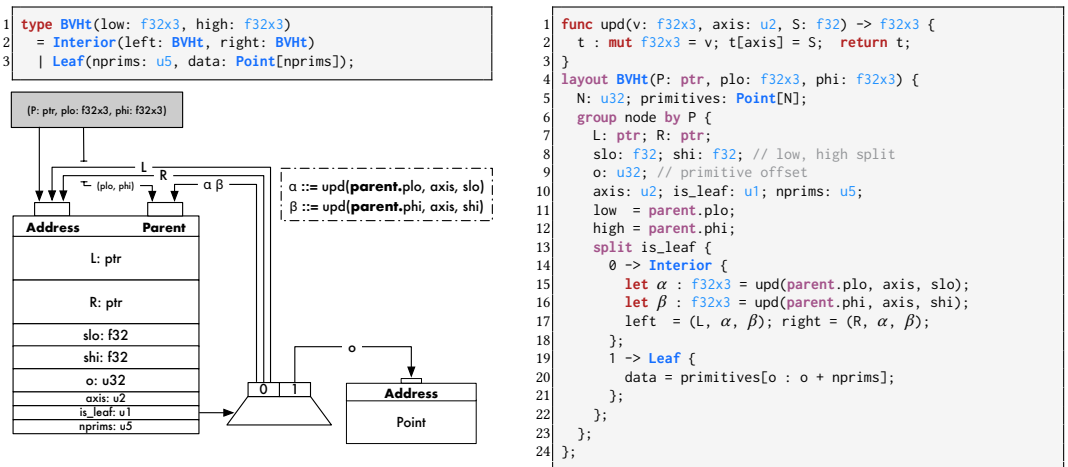


Fig. 14. BVH for the shared slab optimization with tree-carried dependencies specified in SCION.

bounding volumes, remains unchanged, while the physical layout exploits parent-child relationships to reduce memory footprint. This is essential for representing field compression in recursive ADTs.

9 Evaluation

We provide evidence that data representation is a first-order concern in the optimization of tree traversal algorithms over bounding volume hierarchies. We also show that the productivity advantages of SCION do not incur performance overhead when compared to state-of-the-art systems. The evaluation is structured into two parts:

- (1) We conduct an extensive design space exploration demonstrating that Pareto-optimal (performance versus BVH memory footprint) data representations are fundamentally dependent on the characteristics of the input data, target machine, and tree traversal algorithm.
- (2) We position SCION-generated kernels relative to hand-optimized kernels to establish a performance baseline for three applications: ray tracing, closest point query, and collision detection.

9.1 Experimental Methodology

To demonstrate portability, we evaluate SCION on three hardware platforms. The x86 evaluation platform is an Intel Core i9-14900K processor with 24 CPU cores (8 performance, 16 efficiency) and AVX-512 vector extension support. It has a three-level cache hierarchy: 896 KiB of L1 data cache, 32 MiB of L2 cache, and 36 MiB of L3 cache. The ARM evaluation platform is an Apple MacBook M2 Pro with 10 CPU cores, Neon (128-bit) vector extensions, and 16 GiB of unified memory. The GPU evaluation platform is an NVIDIA GeForce RTX 4090 with 24 GiB of GDDR6X memory.

CUDA kernels are compiled with CUDA driver 12.6 and `-O3`; C++ kernels are compiled using Clang 19.1.7 and `-O3 -march=native`. Each benchmark conducts a warm-up run, then executes 9 times, dropping the lowest and highest 2 runs, and reports the weighted average of the remaining 5 runs. Unless stated otherwise, we use a software stack-based traversal (64 entries), consistent with the systems we're comparing to. Implementations for each evaluated algorithm appear in Appendix G: closest-hit ray tracing (CHRT), closest point query (CPQ), and collision detection (CD).

Layouts. Table 1 enumerates the layouts we evaluate using SCION, spanning binary and octonary tree topologies. These layouts draw from established layout optimization techniques [10, 38, 76, 105] while introducing novel compositions and application-specific refinements.

Our evaluation explores four complementary optimization strategies: (1) bounding volume quantization⁵ ranging from 32-bit floating-point to n -bit parent-relative (`-q8`, `-q16`) and world-relative (`-eq`) encodings; (2) 16-byte alignment (`-align16`) to improve memory access alignment; (3) global memory transformation to improve spatial locality during traversal (`-soaos`); and (4) scene-specific optimizations: compressed indexing (`-ci`) exploits the known upper bound on primitives

2-ary Layout	Node Size (B)	Description	8-ary Layout	Node Size (B)	Description
pbvt	32	PBRTv4 LinearBVH [76]	bvh8	256	unoptimized [105]
ptr	48	pbvt + children pointers	bvh8-align16	256	bvh8 + 16B align
pbvt-align16	32	pbvt + 16B align	bvh8-q8	136	8-bit quantized [10]
pbvt-soaos	32	pbvt + SoAoS	bvh8-q8-align16	144	bvh8-q8 + 16B align
pbvt-soaos-align16	32	pbvt-soaos + 16B align	bvh8-q8-ci	104	bvh8-q8 + compressed index
sg-pbvt-q16	16	novel (Section 9.2.5)	bvh8-q8-ci-align16	112	bvh8-q8-ci + 16B align
sg-pbvt-q16-soaos	16	sg-pbvt-q16 + SoAoS	bvh8-q16	184	16-bit quantized
sg-eq	12	snapped grid quantization [38]	bvh8-q16-align16	192	bvh8-q16 + 16B align
sg-eq-align16	16	sg-eq + 16B align	bvh8-q16-ci	152	bvh8-q16 + compressed index
			bvh8-q16-ci-align16	160	bvh8-q16-ci + 16B align

Table 1. Binary (left) and octonary (right) BVH layouts with varying quantization schemes and memory alignment constraints. Node size refers to number of bytes to store a single node in the acceleration structure.

⁵Bounding volume quantization guarantees functional equivalence; each quantized volume fully encloses its original.

(28M triangles in our largest scene) to reduce memory footprint. We exclude the ptr layout from GPU evaluation as pointer-chasing produces a prohibitive slowdown (over two orders of magnitude). Reference implementations of sg-eq and bvh8-q8-ci are provided in Appendix D.

Scenes. Evaluated scenes retrieved from McGuire [66] and Stanford Computer Graphics Laboratory [87] contain triangle counts spanning three orders of magnitude and exhibit diverse geometry ranging from intricate organic models to expansive architectural environments. The only modified scene is san-miguel-x35-y22-z47, which we rotated about its axis by the prescribed degrees (35°, 22°, 47°) so that it is not axis-aligned.

Scene	Triangles
lucy	28,055,728
power-plant	12,759,246
san-miguel-x35-y22-z47	9,832,536
sheep	2,967,664
hairball	2,880,000
sponza	262,267
white-oak	36,760

9.2 Design Space Exploration

We demonstrate that optimal data representations depend fundamentally on three factors: characteristics of the input data (data dependence), the target machine (machine dependence), and the traversal algorithm (algorithm dependence). Using SCION for systematic exploration, we also discover a novel layout that is Pareto-optimal in 35 of 42 evaluation contexts.

9.2.1 Extended Methodology. We evaluate across $2^{\{18,19,20,21,22,23\}}$ rays and compute weighted averages to determine latency (time per ray). We evaluate two ray distributions: (1) *primary rays* originating from cameras and follow coherent paths with high locality, and (2) *secondary rays* arising from light transport simulation, e.g., reflection and refraction, producing divergent, incoherent access patterns. On the CPUs, we parallelize the outer traversal loop using OpenMP dynamic scheduling and a block size of 64. On the GPU, we employ a one-thread-per-ray mapping with 256 threads per block. Presenting the results for the entire Cartesian product of scenes, machines, and ray distributions is impractical, so we present select plots to demonstrate our claims and provide a comprehensive dataset across all scenes, layouts, and platforms in Appendix C.

To isolate the impact of data representation, we standardize several design choices across all evaluation contexts. Every layout uses AABBs, applies explicit indexing into a contiguous primitive array for triangle retrieval, and employs Möller-Trumbore ray-triangle intersection algorithm [70]. Tree construction follows a top-down recursive partitioning scheme with iterative binary refinement, guided by surface area heuristics (SAH) over 32 bins [98].

9.2.2 Data-Dependent. Optimal data representations depend on input data characteristics, including both geometric properties of the indexed primitives and ray distribution patterns. Figure 15 compares Pareto frontiers of the lucy and power-plant scenes, revealing opposite latency orderings: pbvt-q16 outperforms pbvt-q16-soaos by 11% on lucy but underperforms by 9% on power-plant, likely due to differing spatial distributions that affect cache behavior.

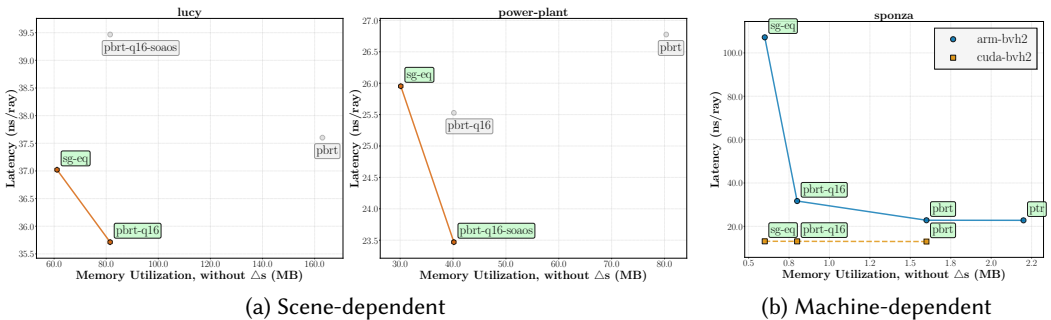


Fig. 15. Performance variation across scenes and architectures for bvh2 layouts. Lines denote the Pareto frontier, and gray points are non-Pareto optimal layouts.

Ray distribution also affects layout performance independently of scene geometry. When tracing *lucy* on the GPU, the same layout yields divergent results for different ray types. Despite using 25% less memory than *pbrt-q16* (the only other Pareto-optimal layout), *sg-eq* exhibits 3.7% slowdown for coherent primary rays and 20.8% slowdown for incoherent secondary rays. Thus, Pareto-optimal layout selection requires consideration of both scene characteristics and ray distribution.

9.2.3 Machine-Dependent. Optimal data layouts also depend on the architecture, reflecting differences in cache hierarchy organization, memory bandwidth characteristics, and instruction set capabilities. We demonstrate this dependence by comparing identical layouts and ray distributions (secondary) across the ARM and GPU architectures with *bvh2* layouts.

As illustrated in Figure 15b, cache hierarchy differences produce divergent layout preferences across platforms. On ARM, eliminating index arithmetic operations in the traversal hot loop reduces memory-dependent operations, enabling *ptr* to marginally outperform *pbrt-align16* when the workload is memory latency bound. In contrast, the same *ptr* layout exhibits catastrophic performance degradation on the GPU (over 100× slowdown), as pointer chasing severely underutilizes the available memory bandwidth. Compact layouts like *sg-eq* and *pbrt-q16* achieve superior performance on the GPU by maximizing memory bandwidth utilization and enabling coalesced memory accesses. The performance gap between these two layouts on ARM is particularly notable: *sg-eq*’s dependence on directed rounding intrinsics during dequantization incurs substantial computational overhead.

9.2.4 Algorithm-Dependent. Figure 16 compares the Pareto frontiers of CHRT and CPQ of the San Miguel scene on the GPU. This comparison employs 2^{20} points and rays respectively with *bvh2* layouts. CHRT’s Pareto-optimal layout is *sg-eq*, while CPQ sees better performance with *pbrt* variants. We attribute this to *sg-eq*’s quantization error: while the layout reduces memory footprint, the coarser bounds lead to additional node visits that degrade CPQ’s pruning efficiency. These results demonstrate that the Pareto optimality of layouts can vary for different traversal algorithms.

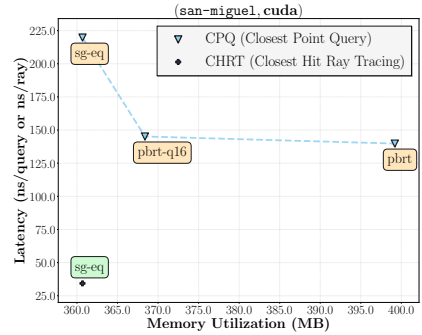


Fig. 16. Algorithm-dependent

9.2.5 Exploring the Layout Design Space: *pbrt-q16*. By decoupling data specification from the logical specification, SCION enables rapid exploration of previously uninvestigated points in the representation design space. Our goal was to discover a layout that achieves Pareto-optimality across diverse evaluation contexts evaluated with our binary BVH layouts. We demonstrate this capability through *pbrt-q16*, a novel layout implemented by combining techniques from disparate sources: implicit indexing from PBRT [76], global coordinate framing from molecular simulation neighbor search [38], and quantization from Benthin et al. [10].

Rationale. PBRTv4 employs an efficient bit-stealing scheme, but requires 32 bytes per node. To reduce the memory footprint, we can explore quantizing the bounding volume, which uses 24 of the 32 bytes. Most existing quantization schemes present a portability–performance tradeoff. For example, the snapped grid extent quantization (*sg-eq*) [38] achieves aggressive compression at 12 bytes per node (a 62.5% reduction from PBRTv4) through 10-bit fixed-point encoding relative to a global coordinate frame anchored at the root. However, *sg-eq* relies on directed rounding modes for their watertight dequantization algorithm, which are expensive on architectures that don’t explicitly support such intrinsics, as discussed in Section 9.2.3.

With a goal of portability, we adopt PBRTv4’s implicit indexing and bit-stealing, then apply 16-bit quantization akin to the 8-bit quantization technique used in Benthin et al. [10]. Unlike Benthin et al. [10], this quantization is applied relative to a global coordinate frame rather than the parent’s frame. This design choice yields three advantages. First, this 16-bit quantization method requires only standard arithmetic operations, eliminating dependence on expensive rounding intrinsics and ensuring portability across CPU and GPU targets. Second, increasing the quantization grid to 2^{16} bins (a $64\times$ increase over sg-eq’s 2^{10} bins) reduces representational errors in bounding volume extents. Consequently, fewer false negative intersections occur. Third, we reduce the node size to 16 bytes by using a global coordinate frame rather than storing the parent AABB and quantized offsets; this aligns naturally with cache line boundaries on contemporary architectures. We provide the full layout and build specification for pbrt-q16 in Appendix D.3.

Results. Among bvh2 layouts, pbrt-q16 achieves Pareto optimality in 35 of 42 evaluation contexts spanning three architectures, seven scenes, and two ray distribution patterns. Notably, 4 of the 7 non-dominating instances (shown in Figure 17) are dominated by pbrt-q16-soaos, a variant of pbrt-q16 that reorganizes the same representation into a struct-of-arrays-of-structs format. While we do not claim Pareto-optimality across all possible design spaces—indeed, the broader thesis of this paper is that no single layout can dominate universally—we demonstrate that systematic composition of orthogonal transformations can yield layouts that are Pareto-optimal within well-defined evaluation contexts.

Machine	Scene	Ray Dist.	Closest Pareto Point
x86	lucy	primary	pbrt-q16-soaos
x86	sponza	secondary	pbrt-q16-soaos
x86	white-oak	secondary	pbrt-q16-soaos
cuda	power-plant	primary	pbrt-q16-soaos
cuda	san-miguel-x35-y22-z47	primary	sg-eq
cuda	sponza	secondary	sg-eq
cuda	white-oak	primary	sg-eq-align16

Fig. 17. The set of (machine $\times$ scene $\times$ ray distribution) contexts where pbrt-q16 is **not** Pareto-optimal.

9.3 Comparison to State-of-the-Art Kernels

Next, we evaluate SCION-generated code against hand-optimized baselines (1) to verify that our abstractions do not incur performance overhead, and (2) to contextualize the results of our design space exploration. While Section 9.2 established data layout as a critical performance factor, the comparison below reveals that scheduling and tree topology yield significant performance variations, demonstrating that layout optimization is one of several orthogonal dimensions in the design space.

9.3.1 Closest-Hit Ray Tracing: Comparison with Embree. We compare (single ray) closest-hit ray tracing written in SCION to Intel Embree [101], the de facto standard for CPU-based ray tracing. Embree represents over a decade of production engineering effort by domain experts and employs sophisticated optimizations beyond data representation, e.g., SIMD-optimized traversal kernels.

Extended Methodology. Our comparison employs Embree’s 8-ary BVH layouts (bvh8i for unquantized, qbv8i for quantized representations). We perform the same experiment as detailed in Section 9.2.1 for Embree and bvh8 layouts. Embree defaults to Plücker coordinates [48] for its

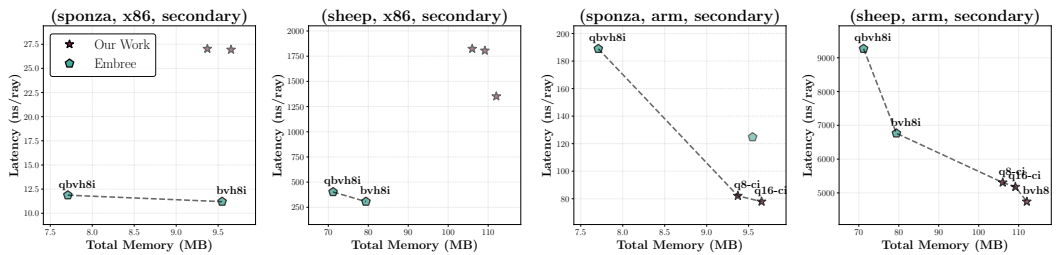


Fig. 18. Closest-hit ray tracing demonstrating (left) worst- and (right) best-case comparison with Embree.

unquantized layout, and uses Möller-Trumbore [70] for its quantized layout. We employ the same intersection methods for our quantized and unquantized bvh8 layouts (described in Table 1). We note two system *input* differences that prevent a direct comparison with Embree. First, while we attempt to approximate Embree’s tree construction strategy, fully replicating all implementation details proved impractical, resulting in different tree topologies with corresponding effects on memory footprint and traversal performance. Second, Embree employs a two-level hierarchy separating geometric instances from primitives, while our system uses a single-level hierarchy directly over primitives. Despite these differences, the comparison demonstrates that decoupling layout from algorithm does not impose prohibitive abstraction overhead relative to hand-optimized kernels.

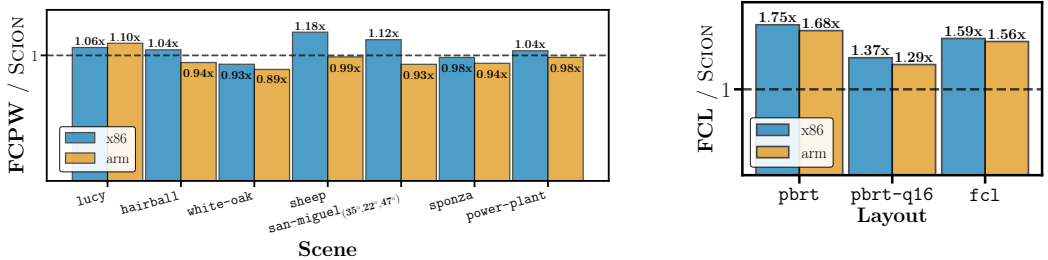
Results. Figure 18 presents CHRT Pareto frontiers on the best- and worst-performing (scene, ray distribution, machine) contexts, selected by averaging performance over Pareto-optimal layouts. The worst-case comparison occurs on x86, an expected result as Embree is developed by Intel. In addition to hand-vectorized tree traversal and intersection, Embree also reduces memory utilization further through aggressive vertex compression. Despite no vectorization and the aforementioned differences in system input, SCION achieves Pareto-optimal performance in 14 of 28 evaluation contexts, three of which are on x86. Results for all contexts (7 scenes $\times$ 2 ray distributions $\times$ 2 architectures) are provided in Appendix E.

9.3.2 Closest Point Query: Comparison with FCPW. The closest point query (CPQ) finds the nearest surface point to a given query position. We compare SCION-generated CPQ against Fastest Closest Points in the West (FCPW) [83], a hand-optimized library for geometric queries.

Extended Methodology. For comparison, we use the same layout as FCPW (pbrt) and implement FCPW’s tree construction algorithm: top-down recursive partitioning with binned surface area heuristic (SAH) splits: primitives are distributed into spatial buckets along each axis, and sweep operations compute partition costs to select optimal splits. Our implementation yields a nearly identical tree (less than 0.0005% node count difference on our largest model, Lucy). We issue 100,000 randomly sampled queries within each scene’s bounding volume, executed single-threaded. Similar to FCPW, we additionally implement distance-based node sorting, which simply visits the children in an order determined by proximity to the point.

Results. Figure 19a illustrates that our performance is comparable to FCPW on both x86 and ARM architectures. Overall, SCION is 4% slower on ARM and 5% faster on x86 (geomean). The worst case (0.89 $\times$ on white-oak) may be explained by two factors: (1) FCPW’s use of highly optimized libraries, e.g., Eigen [32], and (2) SCION’s default check for SENTINEL nodes since some layouts may have invalid children. FCPW assumes sentinels do not exist and elides this check.

9.3.3 Collision Detection: Comparison with FCL. Finally, we implement broad- and narrow-phase collision detection and compare to FCL [74], a widely-deployed collision and proximity library for CPU-based applications.



(a) CPQ speedup vs FCPW (same layout). (b) CD speedup vs FCL (same scene).
Fig. 19. Normalized performance comparison against state-of-the-art libraries (higher is better).

Extended Methodology. We evaluate collisions between two instances of the hairball scene (2.88M triangles each), where one instance is rotated by $(60^\circ, 70^\circ, 10^\circ)$ relative to the other. This configuration produces 5,118,441 colliding triangle pairs, providing substantial coverage of both tree traversal and primitive-level intersection tests. Notably, larger scene collisions resulted in stack overflow due to the recursive nature of FCL’s tree traversal algorithm. To ensure fair comparison, we employ identical construction heuristics and intersection algorithms as FCL. Both systems construct bounding volume hierarchies using median split with one primitive per leaf. Both use the same algorithms for bounding volume overlap tests and employ the Separating Axis Theorem (SAT) for triangle-triangle intersection [91]. This is run single-threaded. Finally, unlike other evaluations, we use recursive function calls during culling to match FCL’s traversal strategy.

Results. Figure 19b shows collision detection speedup relative to FCL; SCION is 1.68× faster on ARM and 1.75× faster on x86 (geomean). We observe two key results: first, when SCION employs the same memory layout as FCL, we achieve modest speedups across platforms. We attribute this improvement to reduced conditional branching overhead: FCL’s implementation includes instrumentation for gathering statistics that adds additional branching to the traversal hot path. Second, when SCION employs optimized layouts originally designed for ray tracing, we observe further speedups (1.29 – 1.75×), demonstrating that some layout optimizations can be effective across traversal algorithms.

10 Related Work

Our work draws on several research threads in programming languages and compilers: fine-grain memory layout control, coarse-grain memory layout control, and separation of algorithm from representation. We position SCION relative to each area.

Fine-Grain Memory Layout Control. Recent work provides programmer control over memory layout at the granularity of individual fields and bits. RIBBIT [6] introduces tagged unions with bit-stealing, enabling bit-accurate, composable discriminated unions. QuanTaichi [40] supports per-field and shared exponent quantization in physical simulation software, trading precision for memory bandwidth. Virgil [90] enables customization of object layouts in an object-oriented, functional language. Dargent [15] applies verified data layout refinement to systems programming, providing formal guarantees about the correspondence between abstract specifications and concrete memory layouts. Many functional languages provide unboxed types to eliminate allocation overhead for small values [34, 44, 59]. Each of these systems operates at the level of individual elements, lacking control over coarse-grain transformations that we provide through reference types.

Coarse-Grain Memory Layout Control. Prior work also enables coarse-grain transformations such as serializing ADTs into flat layouts [94, 95] and transforming AoS to SoA via a type declaration [77]. Prior work in databases has explored improving cache utilization by tiling relations into *minipages* [3]. However, each of these systems only supports a predefined set of transformations and lacks fine-grain control, e.g., bit-stealing, necessary for optimizing BVH layouts.

Separation of Algorithm and Representation. More broadly, the principle of separating logical specification from physical implementation has deep roots in both databases and programming languages. Database systems achieve data independence through query optimization, where logical relational queries compile to diverse physical execution plans [20]. Recent compiler frameworks extend this principle to specialized domains: the Taco compiler [17, 51] decouples sparse tensors from data structure *properties*; GraphIt [107] and Taichi [39] perform a similar decoupling for graphs and spatially sparse grids, respectively. More recently, UniSparse [62] proposed a series of composable transformations for rewriting sparse tensor representations.

11 Conclusion

We present SCION, a system that decouples data representation of bounding volume hierarchies from tree traversal algorithms through two domain-specific languages: a *layout* language for specifying physical memory organization and a *build* language for realizing the transformation from logical tree to physical representation. Through this bidirectional mapping, SCION automatically specializes build and traversal code to arbitrary layouts, enabling systematic exploration of the data representation design space across different algorithms, machines, and data characteristics.

Data Availability Statement

Performance results were generated using a publicly available artifact [33] containing all benchmarking code, scripts, and instructions for reproducibility. Benchmarking results may vary across hardware platforms, which is precisely the thesis of this work.

Acknowledgments

We thank our reviewers for their valuable feedback. We are especially grateful for Benjamin Driscoll's assistance in formalizing the language, and thank Andrew Adams for early feedback on the initial development of the layout language. We also thank Scott Kovach, Shiv Sundram, Olivia Hsu, James Dong, Anderson Truong, and Katherine Mohr for feedback on early drafts of this paper. Finally, we extend special thanks to Rohan Yadav, who, by sheer misfortune, found himself seated in close proximity to the authors and graciously endured endless questions about scientific prose.

We acknowledge Martin Káčerik for access to sheep; Morgan McGuire's Computer Graphics Archive [66] for access to hairball, san-miguel, power-plant, white-oak, and sponza; and Stanford University Computer Graphics Laboratory [87] for access to lucy. Christophe and Alexander were supported by the Qualcomm Innovation Fellowship and SystemX Alliance. This work was supported in part by the NSF under grant numbers 2216964 and 2143061, by the Stanford Portal Center, and by PRISM, one of seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

References

- [1] Maaz Bin Safeer Ahmad, Alexander J Root, Andrew Adams, Shoaib Kamil, and Alvin Cheung. 2022. Vector instruction selection for digital signal processors using program synthesis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '22). Association for Computing Machinery, New York, NY, USA, 1004–1016. doi:10.1145/3503222.3507714
- [2] Timo Aila and Samuli Laine. 2009. Understanding the efficiency of ray traversal on GPUs. In *Proceedings of the conference on high performance graphics 2009*. 145–149.
- [3] Anastassia Ailamaki, David J. DeWitt, and Mark D. Hill. 2002. Data page layouts for relational databases on deep memory hierarchies. *The VLDB Journal* 11, 3 (Nov. 2002), 198–215. doi:10.1007/s00778-002-0074-9
- [4] Ciprian Apetrei. 2014. Fast and simple agglomerative LBVH construction. (2014).
- [5] Wilhem Barbier and Mathias Paulin. 2025. Fused Collapsing for Wide BVH Construction. In *Computer Graphics Forum*. Wiley Online Library, e70213.
- [6] Thaïs Baudon, Gabriel Radanne, and Laure Gonnord. 2023. Bit-Stealing Made Legal: Compilation for Custom Memory Representations of Algebraic Data Types. *Proc. ACM Program. Lang.* 7, ICFP, Article 216 (Aug. 2023), 34 pages. doi:10.1145/3607858
- [7] Pablo Bauszat, Martin Eisemann, and Marcus A Magnor. 2010. The Minimal Bounding Volume Hierarchy.. In *VMV*.
- [8] Carsten Benthin, Daniel Meister, Joshua Barczak, Rohan Mehalwal, John Tsakok, and Andrew Kensler. 2024. H-PLOC: Hierarchical Parallel Locally-Ordered Clustering for Bounding Volume Hierarchy Construction. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 7, 3 (2024), 1–14.
- [9] Carsten Benthin, Karthik Vaidyanathan, and Sven Woop. 2021. Ray Tracing Lossy Compressed Grid Primitives.. In *Eurographics (Short Papers)*. 1–4.
- [10] Carsten Benthin, Ingo Wald, Sven Woop, and Attila T. Áfra. 2018. Compressed-leaf bounding volume hierarchies. In *Proceedings of the Conference on High-Performance Graphics* (Vancouver, British Columbia, Canada) (HPG '18). Association for Computing Machinery, New York, NY, USA, Article 6, 4 pages. doi:10.1145/3231578.3231581

- [11] Carsten Benthin, Sven Woop, Ingo Wald, and Attila T Áfra. 2017. Improved two-level BVHs using partial re-braiding. In *Proceedings of High Performance Graphics*. 1–8.
- [12] Solomon Boulos, Dave Edwards, J. Dylan Lacewell, Joe Kniss, Jan Kautz, Peter Shirley, and Ingo Wald. 2007. Packet-based whitted and distribution ray tracing. In *Proceedings of Graphics Interface 2007* (Montreal, Canada) (GI '07). Association for Computing Machinery, New York, NY, USA, 177–184. doi:10.1145/1268517.1268547
- [13] Martin Burtscher and Keshav Pingali. 2011. An efficient CUDA implementation of the tree-based Barnes-Hut N-body algorithm. In *GPU computing Gems Emerald edition*. Elsevier, 75–92.
- [14] Yishen Chen, Charith Mendis, Michael Carbin, and Saman Amarasinghe. 2021. VeGen: a vectorizer generator for SIMD and beyond. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 902–914. doi:10.1145/3445814.3446692
- [15] Zilin Chen, Ambroise Lafont, Liam O'Connor, Gabriele Keller, Craig McLaughlin, Vincent Jackson, and Christine Rizkallah. 2023. Dargent: A Silver Bullet for Verified Data Layout Refinement. *Proc. ACM Program. Lang.* 7, POPL, Article 47 (Jan. 2023), 27 pages. doi:10.1145/3571240
- [16] Floyd M Chitalu, Christophe Dubach, and Taku Komura. 2020. Binary Ostensibly-Implicit Trees for Fast Collision Detection. In *Computer Graphics Forum*, Vol. 39. Wiley Online Library, 509–521.
- [17] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format abstraction for sparse tensor algebra compilers. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 123 (Oct. 2018), 30 pages. doi:10.1145/3276493
- [18] Per Christensen, Julian Fong, Charlie Kilpatrick, Francisco González, Srinath Ravichandran, Akshay Shah, Ethan Jaszewski, Stephen Friedman, James Burgess, Trina M Roy, et al. 2025. RenderMan XPU: A Hybrid CPU+ GPU Renderer for Interactive and Final-frame Rendering. In *COMPUTER GRAPHICS forum*, Vol. 44.
- [19] David Cline, Kevin Steele, and Parris Egbert. 2006. Lightweight bounding volumes for ray tracing. *Journal of Graphics Tools* 11, 4 (2006), 61–71.
- [20] E. F. Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (June 1970), 377–387. doi:10.1145/362384.362685
- [21] Jonathan D Cohen, Ming C Lin, Dinesh Manocha, and Madhav Ponamgi. 1995. I-collide: An interactive and exact collision detection system for large-scale environments. In *Proceedings of the 1995 symposium on Interactive 3D graphics*. 189–ff.
- [22] Daniel S Coming and Oliver G Staadt. 2007. Velocity-aligned discrete oriented polytopes for dynamic collision detection. *IEEE Transactions on Visualization and Computer Graphics* 14, 1 (2007), 1–12.
- [23] Holger Dammert, Johannes Hanika, and Alexander Keller. 2008. Shallow bounding volume hierarchies for fast SIMD ray tracing of incoherent rays. In *Computer Graphics Forum*, Vol. 27. Wiley Online Library, 1225–1233.
- [24] David Eberly. 2002. Dynamic collision detection using oriented bounding boxes. *Geometric Tools, Inc* (2002).
- [25] Martin Eisemann, Pablo Bauszat, and Marcus Magnor. 2012. Implicit object space partitioning: The no-memory BVH. *Comput. Graph. Braunsch.* (2012).
- [26] Martin Eisemann, Christian Woizischke, and Marcus A Magnor. 2008. Ray Tracing with the Single Slab Hierarchy. In *VMV*. 373–381.
- [27] Christer Ericson. 2004. *Real-Time Collision Detection*. CRC Press, Inc., USA.
- [28] Manfred Ernst and Gunther Greiner. 2008. Multi bounding volume hierarchies. In *2008 IEEE Symposium on Interactive Ray Tracing*. 35–40. doi:10.1109/RT.2008.4634618
- [29] Jeffrey Goldsmith and John Salmon. 1987. Automatic creation of object hierarchies for ray tracing. *IEEE Computer Graphics and Applications* 7, 5 (1987), 14–20.
- [30] Stefan Gottschalk, Ming C Lin, and Dinesh Manocha. 1996. OBBTree: A hierarchical structure for rapid interference detection. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*. 171–180.
- [31] Yan Gu, Yong He, Kayvon Fatahalian, and Guy Blueloch. 2013. Efficient BVH construction via approximate agglomerative clustering. In *Proceedings of the 5th High-Performance Graphics Conference*. 81–88.
- [32] Gaël Guennebaud, Benoît Jacob, et al. 2010. *Eigen v3*.
- [33] Christophe Gyurgyik, Alexander J Root, and Fredrik Kjolstad. 2026. Artifact for 'Decoupling Data Layouts from Bounding Volume Hierarchies'. Zenodo. doi:10.5281/zenodo.19009103
- [34] Cordelia Hall, Simon L Peyton Jones, and Patrick M Sansom. 1994. Unboxing using specialisation. In *Functional Programming, Glasgow 1994: Proceedings of the 1994 Glasgow Workshop on Functional Programming*, Ayr, Scotland, 12–14 September 1994. Springer, 96–110.
- [35] Vlastimil Havran, Robert Herzog, and Hans-peter Seidel. 2006. On the Fast Construction of Spatial Hierarchies for Ray Tracing. In *2006 IEEE Symposium on Interactive Ray Tracing*. 71–80. doi:10.1109/RT.2006.280217
- [36] Roger Hindley. 1969. The principal type-scheme of an object in combinatory logic. *Transactions of the american mathematical society* 146 (1969), 29–60.

- [37] Michael P Howard, Joshua A Anderson, Arash Nikoubashman, Sharon C Glotzer, and Athanassios Z Panagiotopoulos. 2016. Efficient neighbor list calculation for molecular simulation of colloidal systems using graphics processing units. *Computer Physics Communications* 203 (2016), 45–52.
- [38] Michael P Howard, Antonia Statt, Felix Madutsa, Thomas M Truskett, and Athanassios Z Panagiotopoulos. 2019. Quantized bounding volume hierarchies for neighbor search in molecular simulations on graphics processing units. *Computational Materials Science* 164 (2019), 139–146.
- [39] Yuanming Hu, Tzu-Mao Li, Luke Anderson, Jonathan Ragan-Kelley, and Frédo Durand. 2019. Taichi: a language for high-performance computation on spatially sparse data structures. *ACM Transactions on Graphics (TOG)* 38, 6 (2019).
- [40] Yuanming Hu, Jiafeng Liu, Xuanda Yang, Mingkuan Xu, Ye Kuang, Weiwei Xu, Qiang Dai, William T Freeman, and Frédo Durand. 2021. Quantaichi: a compiler for quantized simulations. *ACM Transactions on Graphics (TOG)* 40, 4 (2021).
- [41] Yen-Chieh Huang, Chen-Pin Yang, and Tsung Tai Yeh. 2025. AQB8: Energy-Efficient Ray Tracing Accelerator through Multi-Level Quantization. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*.
- [42] Philip Martyn Hubbard. 2002. Collision detection for interactive graphics applications. *IEEE Transactions on Visualization and Computer Graphics* 1, 3 (2002), 218–230.
- [43] Yuka Ikarashi, Gilbert Louis Bernstein, Alex Reinking, Hasan Genc, and Jonathan Ragan-Kelley. 2022. Exocompilation for productive programming of hardware accelerators. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) (PLDI 2022). Association for Computing Machinery, New York, NY, USA, 703–718. doi:10.1145/3519939.3523446
- [44] Simon L Peyton Jones and John Launchbury. 1991. Unboxed values as first class citizens in a non-strict functional language. In *Conference on Functional Programming Languages and Computer Architecture*. Springer, 636–666.
- [45] Martin Káčerik and Jiri Bittner. 2024. SAH-Optimized k-DOP Hierarchies for Ray Tracing. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 7, 3 (2024).
- [46] M Káčerik and Jiri Bittner. 2025. SOBB: Skewed Oriented Bounding Boxes for Ray Tracing. In *Computer Graphics Forum*. Wiley Online Library, e70062.
- [47] Tero Karras. 2012. Maximizing parallelism in the construction of BVHs, octrees, and k-d trees. In *Proceedings of the Fourth ACM SIGGRAPH/Eurographics Conference on High-Performance Graphics*. 33–37.
- [48] Timothy L. Kay and James T. Kajiya. 1986. Ray Tracing Complex Scenes. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*. 269–278. doi:10.1145/15922.15916
- [49] Dong Jin Kim, Leonidas J Guibas, and Sung Yong Shin. 1997. Fast collision detection among multiple moving spheres. In *Proceedings of the thirteenth annual symposium on Computational geometry*. 373–375.
- [50] Tae-Joon Kim, Bochang Moon, Duksu Kim, and Sung-Eui Yoon. 2010. RACBVHs: Random-Accessible Compressed Bounding Volume Hierarchies. *IEEE Transactions on Visualization and Computer Graphics* 16, 2 (2010), 273–286. doi:10.1109/TVCG.2009.71
- [51] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The tensor algebra compiler. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–29.
- [52] James T Klosowski, Martin Held, Joseph SB Mitchell, Henry Sowizral, and Karel Zikan. 1998. Efficient collision detection using bounding volume hierarchies of k-DOPs. *IEEE transactions on Visualization and Computer Graphics* 4, 1 (1998), 21–36.
- [53] Sergey Kosarevsky, Roman Kuznetsov, and Alexey Medvedev. 2025. Ray Tracing with Bindless Vulkan on Mobile Devices, a Case Study: Performance. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '25)*. Association for Computing Machinery, New York, NY, USA, Article 11, 2 pages. doi:10.1145/3721239.3734103
- [54] Samuli Laine, Tero Karras, and Timo Aila. 2013. Megakernels considered harmful: Wavefront path tracing on GPUs. In *Proceedings of the 5th High-Performance Graphics Conference*. 137–143.
- [55] Thomas Larsson and Tomas Akenine-Möller. 2009. Bounding volume hierarchies of slab cut balls. In *Computer Graphics Forum*, Vol. 28. Wiley Online Library, 2379–2395.
- [56] Christian Lauterbach, Michael Garland, Shubhabrata Sengupta, David Luebke, and Dinesh Manocha. 2009. Fast BVH construction on GPUs. In *Computer Graphics Forum*, Vol. 28. Wiley Online Library, 375–384.
- [57] Christian Lauterbach, Sung-eui Yoon, Ming Tang, and Dinesh Manocha. 2008. ReduceM: Interactive and memory efficient ray tracing of large models. In *Computer Graphics Forum*, Vol. 27. Wiley Online Library, 1313–1321.
- [58] Won-Jong Lee, Youngsam Shin, Jaedon Lee, Jin-Woo Kim, Jae-Ho Nah, Seokyeon Jung, Shihwa Lee, Hyun-Sang Park, and Tack-Don Han. 2013. SGRIT: a mobile GPU architecture for real-time ray tracing. In *Proceedings of the 5th High-Performance Graphics Conference (Anaheim, California) (HPG '13)*. Association for Computing Machinery, New York, NY, USA, 109–119. doi:10.1145/2492045.2492057
- [59] Xavier Leroy. 1992. Unboxed objects and polymorphic typing. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Albuquerque, New Mexico, USA) (POPL '92). Association for

- Computing Machinery, New York, NY, USA, 177–188. doi:10.1145/143165.143205
- [60] Gábor Liktó and Karthikeyan Vaidyanathan. 2016. Bandwidth-efficient BVH layout for incremental hardware traversal. In *High Performance Graphics*. 51–61.
 - [61] Daqi Lin, Elena Vasiou, Cem Yuksel, Daniel Kopta, and Erik Brunvand. 2020. Hardware-accelerated dual-split trees. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 3, 2 (2020), 1–21.
 - [62] Jie Liu, Zhongyuan Zhao, Zijian Ding, Benjamin Brock, Hongbo Rong, and Zhiru Zhang. 2024. UniSparse: An Intermediate Language for General Sparse Format Customization. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 99 (April 2024), 29 pages. doi:10.1145/3649816
 - [63] Lufei Liu, Mohammadreza Saed, Yuan Hsi Chou, Davit Grigoryan, Tyler Nowicki, and Tor M Aamodt. 2023. LumiBench: A benchmark suite for hardware ray tracing. In *2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 1–14.
 - [64] Jeffrey Mahovsky and Brian Wyvill. 2006. Memory-conserving bounding volume hierarchies with coherent raytracing. In *Computer Graphics Forum*, Vol. 25. Wiley Online Library, 173–182.
 - [65] Jeffrey A Mahovsky. 2005. *Ray tracing with reduced-precision bounding volume hierarchies*. Ph.D. Dissertation. University of Calgary.
 - [66] Morgan McGuire. 2017. Computer Graphics Archive. <https://casual-effects.com/data>
 - [67] Daniel Meister and Jiri Bittner. 2017. Parallel locally-ordered clustering for bounding volume hierarchy construction. *IEEE transactions on visualization and computer graphics* 24, 3 (2017), 1345–1353.
 - [68] Daniel Meister, Jakub Boksansky, Michael Guthe, and Jiri Bittner. 2020. On Ray Reordering Techniques for Faster GPU Ray Tracing. In *Symposium on Interactive 3D Graphics and Games (San Francisco, CA, USA) (I3D '20)*. Association for Computing Machinery, New York, NY, USA, Article 13, 9 pages. doi:10.1145/3384382.3384534
 - [69] Daniel Meister, Shinji Ogaki, Carsten Benthin, Michael J Doyle, Michael Guthe, and Jiri Bittner. 2021. A survey on bounding volume hierarchies for ray tracing. In *Computer Graphics Forum*, Vol. 40. Wiley Online Library, 683–712.
 - [70] Tomas Möller and Ben Trumbore. 1997. Fast, Minimum Storage Ray/Triangle Intersection. *Journal of Graphics Tools* 2, 1 (1997), 21–28. doi:10.1080/10867651.1997.10487468
 - [71] Benjamin Mora. 2011. Naive ray-tracing: A divide-and-conquer approach. *ACM Trans. Graph.* 30, 5, Article 117 (Oct. 2011), 12 pages. doi:10.1145/2019627.2019636
 - [72] Jae-Ho Nah, Hyuck-Joo Kwon, Dong-Seok Kim, Cheol-Ho Jeong, Jinhong Park, Tack-Don Han, Dinesh Manocha, and Woo-Chan Park. 2014. RayCore: A Ray-Tracing Hardware Architecture for Mobile Devices. *ACM Trans. Graph.* 33, 5, Article 162 (Sept. 2014), 15 pages. doi:10.1145/2629634
 - [73] Ian J. Palmer and Richard L. Grimsdale. 1995. Collision detection for animation using sphere-trees. In *Computer Graphics Forum*, Vol. 14. Wiley Online Library, 105–116.
 - [74] Jia Pan, Sachin Chitta, and Dinesh Manocha. 2012. FCL: A general purpose library for collision and proximity queries. In *2012 IEEE International Conference on Robotics and Automation*. 3859–3866. doi:10.1109/ICRA.2012.6225337
 - [75] Jacopo Pantaleoni and David Luebke. 2010. HLBVH: Hierarchical LBVH construction for real-time ray tracing of dynamic geometry. In *Proceedings of the Conference on High Performance Graphics*. 87–95.
 - [76] Matt Pharr, Wenzel Jakob, and Greg Humphreys. 2023. *Physically based rendering: From theory to implementation*. MIT Press.
 - [77] Matt Pharr and William R. Mark. 2012. ispc: A SPMD compiler for high-performance CPU programming. In *2012 Innovative Parallel Computing (InPar)*. 1–13. doi:10.1109/InPar.2012.6339601
 - [78] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (Seattle, Washington, USA) (PLDI '13)*. Association for Computing Machinery, New York, NY, USA, 519–530. doi:10.1145/2491956.2462176
 - [79] Henry Gordon Rice. 1953. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical society* 74, 2 (1953), 358–366.
 - [80] Alexander J Root, Christophe Gyurgyik, Purvi Goel, Kayvon Fatahalian, Jonathan Ragan-Kelley, Andrew Adams, and Fredrik Kjolstad. 2026. Bonsai: Compiling Queries to Pruned Tree Traversals. *Proceedings of the ACM on Programming Languages* 10, PLDI, Article 178 (June 2026). doi:10.1145/3808256
 - [81] Steven M. Rubin and Turner Whitted. 1980. A 3-dimensional representation for fast rendering of complex scenes. *SIGGRAPH Comput. Graph.* 14, 3 (July 1980), 110–116. doi:10.1145/965105.807479
 - [82] Mohammadreza Saed, Yuan Hsi Chou, Lufei Liu, Tyler Nowicki, and Tor M Aamodt. 2022. Vulkan-Sim: A GPU architecture simulator for ray tracing. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 263–281.
 - [83] Rohan Sawhney. 2021. *FCPW: Fastest Closest Points in the West*.

- [84] Benjamin Segovia and Manfred Ernst. 2010. Memory efficient ray tracing with hierarchical mesh quantization. In *Proceedings of Graphics Interface 2010* (Ottawa, Ontario, Canada) (GI '10). Canadian Information Processing Society, CAN, 153–160.
- [85] Woong Seo, Yeonsoo Kim, and Insung Ihm. 2017. Effective ray tracing of large 3D scenes through mobile distributed computing. In *SIGGRAPH Asia 2017 Mobile Graphics & Interactive Applications* (Bangkok, Thailand) (SA '17). Association for Computing Machinery, New York, NY, USA, Article 3, 5 pages. doi:10.1145/3132787.3139206
- [86] Brian Smits. 2005. Efficiency issues for ray tracing. In *ACM SIGGRAPH 2005 Courses* (Los Angeles, California) (SIGGRAPH '05). Association for Computing Machinery, New York, NY, USA, 6–es. doi:10.1145/1198555.1198745
- [87] Stanford Computer Graphics Laboratory. 2023. The Stanford 3D Scanning Repository. Online. <https://graphics.stanford.edu/data/3Dscanrep/> Accessed: November 2025.
- [88] Christopher Strachey. 2000. Fundamental concepts in programming languages. *Higher-order and symbolic computation* 13 (2000), 11–49.
- [89] Wolfgang Stürzlinger and Robert Tobler. 1994. Two Optimization Methods for Ray Tracing. In *Spring School on Computer Graphics (SCCG '94)*. 104–107.
- [90] Bradley Wei Jie Teo and Ben L. Titzer. 2024. Unboxing Virgil ADTs for Fun and Profit. In *Proceedings of the Workshop Dedicated to Jens Palsberg on the Occasion of His 60th Birthday* (Pasadena, CA, USA) (JENSFEST '24). Association for Computing Machinery, New York, NY, USA, 43–52. doi:10.1145/3694848.3694857
- [91] Oren Tropp, Ayellet Tal, and Ilan Shimshoni. 2006. A fast triangle to triangle intersection test for collision detection. *Computer Animation and Virtual Worlds* 17, 5 (2006), 527–535. doi:10.1002/cav.115
- [92] Karthikeyan Vaidyanathan, Tomas Akenine-Möller, and Marco Salvi. 2016. Watertight ray traversal with reduced precision.. In *High Performance Graphics*. 33–40.
- [93] Alexa VanHattum, Rachit Nigam, Vincent T. Lee, James Bornholt, and Adrian Sampson. 2021. Vectorization for digital signal processors via equality saturation. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 874–886. doi:10.1145/3445814.3446707
- [94] Michael Vollmer, Chaitanya Koparkar, Mike Rainey, Laith Sakka, Milind Kulkarni, and Ryan R Newton. 2019. LoCal: a language for programs operating on serialized data. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 48–62.
- [95] Michael Vollmer, Sarah Spall, Buddhika Chamith, Laith Sakka, Chaitanya Koparkar, Milind Kulkarni, Sam Tobin-Hochstadt, and Ryan R Newton. 2017. Compiling tree transforms to operate on packed representations. (2017).
- [96] Carsten Wächter and Alexander Keller. 2006. Instant ray tracing: The bounding interval hierarchy. *Rendering Techniques* 2006, 139–149 (2006), 130.
- [97] Carsten Wachter and Alexander Keller. 2007. Terminating spatial hierarchies by a priori bounding memory. In *2007 IEEE Symposium on Interactive Ray Tracing*. IEEE, 41–46.
- [98] Ingo Wald. 2007. On fast construction of SAH-based bounding volume hierarchies. In *2007 IEEE Symposium on Interactive Ray Tracing*. IEEE, 33–40.
- [99] Ingo Wald, Carsten Benthin, and Solomon Boulos. 2008. Getting rid of packets - Efficient SIMD single-ray traversal using multi-branching BVHs -. In *2008 IEEE Symposium on Interactive Ray Tracing*. 49–57. doi:10.1109/RT.2008.4634620
- [100] Ingo Wald, Solomon Boulos, and Peter Shirley. 2007. Ray tracing deformable scenes using dynamic bounding volume hierarchies. *ACM Trans. Graph.* 26, 1 (Jan. 2007), 6–es. doi:10.1145/1189762.1206075
- [101] Ingo Wald, Sven Woop, Carsten Benthin, Gregory S Johnson, and Manfred Ernst. 2014. Embree: a kernel framework for efficient CPU ray tracing. *ACM Transactions on Graphics (TOG)* 33, 4 (2014), 1–8.
- [102] Bruce Walter, Kavita Bala, Milind Kulkarni, and Keshav Pingali. 2008. Fast agglomerative clustering for rendering. In *2008 IEEE Symposium on Interactive Ray Tracing*. IEEE, 81–86.
- [103] Hank Weghorst, Gary Hooper, and Donald P Greenberg. 1984. Improved computational methods for ray tracing. *ACM Transactions on Graphics (TOG)* 3, 1 (1984), 52–69.
- [104] Dominik Wodniok, André Schulz, Sven Widmer, and Michael Goesele. 2013. Analysis of Cache Behavior and Performance of Different BVH Memory Layouts for Tracing Incoherent Rays.. In *EGPGV@ Eurographics*. 57–64.
- [105] Sven Woop, Carsten Benthin, Ingo Wald, Gregory S Johnson, and Eric Tabellion. 2014. Exploiting Local Orientation Similarity for Efficient Ray Traversal of Hair and Fur. *High Performance Graphics* 3 (2014).
- [106] Henri Ylitie, Tero Karras, and Samuli Laine. 2017. Efficient incoherent ray traversal on GPUs through compressed wide BVHs. In *Proceedings of High Performance Graphics* (Los Angeles, California) (HPG '17). Association for Computing Machinery, New York, NY, USA, Article 4, 13 pages. doi:10.1145/3105762.3105773
- [107] Yunming Zhang, Mengjiao Yang, Riyadh Baghdadi, Shoaib Kamil, Julian Shun, and Saman Amarasinghe. 2018. GraphIt: a high-performance graph DSL. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 121 (Oct. 2018), 30 pages. doi:10.1145/3276491

Appendix

A Algorithm for Layout Path Uniqueness

Algorithm 2 Checking uniqueness of paths for field F of variant type T_v .

```

1:  $M$ , layout member to traverse;  $F$ , target field to resolve;
2:  $T_v$ , current variant type for SPLIT disambiguation
3:  $I$ , map of indirect group identifier to layouts
4: function COUNTPATHS( $M, F, T_v$ )
5:   match  $M$  with
6:   | FIELD( $id$ )  $\Rightarrow$ 
7:   | DERIVE( $id, E$ )  $\Rightarrow$ 
8:     if  $id = F$  then return 1
9:   | else return 0
10:  | GROUP( $T_G, body, index$ )  $\Rightarrow$ 
11:    match  $T_G$  with
12:    | DIRECT  $\Rightarrow$  return COUNTPATHS( $body, F, T_v, I$ )
13:    | INDIRECT( $name$ )  $\Rightarrow$  insert ( $name, body$ ) into  $I$ ; return 0
14:  | FROM( $name, key$ )  $\Rightarrow$ 
15:    let  $body \leftarrow I[name]$  in ▷ Fail if  $name \notin I$ 
16:    return COUNTPATHS( $body, F, T_v, I$ )
17:  | SPLIT( $discriminant, arms$ )  $\Rightarrow$ 
18:    for each  $Arm(C, T_a, body) \in arms$  do
19:      if  $T_a = T_v$  then
20:        then return COUNTPATHS( $body, F, T_v, I$ )
21:  | SEQUENCE( $layouts$ )  $\Rightarrow$ 
22:    let  $c \leftarrow 0$  in
23:    for each  $L \in layouts$  do
24:       $c \leftarrow c + \text{COUNTPATHS}(L, F, T_v, I)$ 
25:    if  $c > 1$  then fail ▷ Multiple paths to field  $F$ 
26:    else return  $c$ 
27: end function

```

B Well-Formedness of the Derivation-Free Fragment

The *derivation-free fragment* is a subset of the SCION languages where all fields are stored; this disallows the use of relational primitives and derived fields. We assume the reference type $\mathcal{T}_\ell$ is injective, so distinct logical elements produce distinct references and therefore disjoint store regions. We state the well-formedness conditions summarized in Section 6 for this subset of SCION, proving layout completeness (Theorem B.4) and providing a proof sketch for round-trip soundness (Theorem B.5).

B.1 Logical Representation

A *named field* is a pair (x, T) of an identifier x and a type T , written $x : T$. A *named variant* is a pair $(V, \overline{x : \overline{T}})$ of a constructor name V and a finite sequence of distinctly named fields, written $V(\overline{x : \overline{T}})$. We formalize over recursive algebraic data types of the form:

$$\begin{aligned} \tau &::= \mu\alpha. V_1(\overline{x_1 : \overline{T}_1}) + \dots + V_m(\overline{x_m : \overline{T}_m}) \\ T &::= t \mid \alpha \mid T_1 \times \dots \times T_n \mid [T] \end{aligned}$$

where t ranges over primitive types and α is the recursive variable bound by μ . The surface language permits arbitrarily nested sum types. For the formalization, we elaborate each source type τ^* to a sum-of-products normal form $\text{NF}(\tau^*)$ by hoisting nested sums to fresh top-level variants. The formal development below ranges over $\tau = \text{NF}(\tau^*)$. We write $\mathcal{V}(\tau)$ for the set of variants, and

$$\mathcal{N}(V) = \{(x, T) \mid x : T \text{ is a named field of } V\}$$

for the named fields of variant V . Let $[\cdot]_\ell$ denote the substitution $[\alpha \mapsto \mathcal{T}_\ell]$ applied structurally, where $\mathcal{T}_\ell$ is the reference type declared by layout ℓ :

$$[t]_\ell = t \quad [\alpha]_\ell = \mathcal{T}_\ell \quad [T_1 \times \dots \times T_n]_\ell = [T_1]_\ell \times \dots \times [T_n]_\ell \quad [[T]]_\ell = [[T]_\ell]$$

Finally, we define the *resolved field set* of variant V for layout ℓ as the following:

$$\mathcal{F}_\ell(V) = \{(x, [T]_\ell) \mid (x, T) \in \mathcal{N}(V)\}.$$

B.2 Physical Paths

In the derivation-free fragment, layout members have the grammar:

$$M ::= (x : T) \mid \text{group } g \text{ by } I \{M\} \mid \text{split } e \{C_i \rightarrow V_i\{M_i\}\}_{i=1}^n \mid M_0; M_1$$

A *physical path* records how a terminal (stored field) is reached from the root reference:

$$\pi ::= \varepsilon \mid \mathcal{G}(g, I) \cdot \pi \mid \mathcal{S}(e, V) \cdot \pi$$

where $\mathcal{G}(g, I)$ addresses group g via I , and $\mathcal{S}(e, V)$ selects the arm labeled V based on discriminant e . The judgment $V \vdash M : (\pi, x : T)$ states that terminal $x : T$ is reachable in member M by path π when decoding variant V :

$$\frac{}{V \vdash (x : T) : (\varepsilon, x : T)} \quad (\text{P-FLD})$$

$$\frac{V \vdash M : (\pi, x : T)}{V \vdash \text{group } g \text{ by } I \{M\} : (\mathcal{G}(g, I) \cdot \pi, x : T)} \quad (\text{P-GRP})$$

$$\frac{V_j = V \quad V \vdash M_j : (\pi, x : T)}{V \vdash \text{split } e \{C_i \rightarrow V_i\{M_i\}\}_{i=1}^n : (\mathcal{S}(e, V) \cdot \pi, x : T)} \quad (\text{P-ARM})$$

$$\frac{V \vdash M_k : (\pi, x : T) \quad k \in \{0, 1\}}{V \vdash M_0; M_1 : (\pi, x : T)} \quad (\text{P-SEQ})$$

P-FLD terminates at a named field. P-GRP prepends a group step. P-ARM selects the arm $V_j = V$ and prepends a split step. P-SEQ propagates into either member without modifying the path.

B.3 Layout Well-Formedness

Definition B.1 (Layout completeness). Let $\mathcal{P}(V, \ell) = \{(\pi, x : T) \mid V \vdash M_\ell : (\pi, x : T)\}$, where M_ℓ is the body of layout ℓ . A layout ℓ is *complete* if, for every $V \in \mathcal{V}(\tau)$ and every $(x, T) \in \mathcal{F}_\ell(V)$, there exists exactly one $(\pi, x : T) \in \mathcal{P}(V, \ell)$.

Completeness says every logical named field has exactly one unambiguous physical path with the correct resolved type. A scope environment Γ is a finite map from names to scope classes:

$$\Gamma ::= x : T \mid g : \text{grp}.$$

Entries of the form $x : T$ encompass both reference parameters and field names, while $g : \text{grp}$ are group names. A *terminal binding set* Δ is a finite set of typed bindings $\{x : T\}$. The judgment $\Gamma; V \vdash M \text{ wf} : \Delta$ states that member M is well-formed in scope Γ for variant V and exposes terminal bindings Δ .

$$\frac{x \notin \text{dom}(\Gamma) \quad (x, U) \in \mathcal{N}(V) \implies T = \lfloor U \rfloor_\ell}{\Gamma; V \vdash (x : T) \text{ wf} : \{x : T\}} \quad (\text{WF-FLD})$$

$$\frac{(I : T) \in \Gamma \quad g \notin \text{dom}(\Gamma) \quad (\Gamma, g : \text{grp}); V \vdash M \text{ wf} : \Delta}{\Gamma; V \vdash \text{group } g \text{ by } I \{M\} \text{ wf} : \Delta} \quad (\text{WF-GRP})$$

$$\frac{(e : T) \in \Gamma \quad \bigsqcup_{i=1}^n C_i = \llbracket T \rrbracket \quad \exists! j. V_j = V \quad \Gamma; V \vdash M_j \text{ wf} : \Delta}{\Gamma; V \vdash \text{split } e \{C_i \rightarrow V_i\{M_i\}\}_{i=1}^n \text{ wf} : \Delta} \quad (\text{WF-SPLIT})$$

$$\frac{\Gamma; V \vdash M_0 \text{ wf} : \Delta_0 \quad (\Gamma, \Delta_0); V \vdash M_1 \text{ wf} : \Delta_1}{\Gamma; V \vdash M_0; M_1 \text{ wf} : \Delta_0 \cup \Delta_1} \quad (\text{WF-SEQ})$$

$$\frac{\forall V \in \mathcal{V}(\tau). \quad \Gamma_0; V \vdash M_\ell \text{ wf} : \Delta_V \quad \mathcal{F}_\ell(V) \subseteq \Delta_V}{\vdash \ell \text{ wf}} \quad (\text{WF-LAYOUT})$$

WF-FLD registers a fresh terminal. When x names a logical field of V , the type must match. WF-GRP requires the address $I : T$ is in scope, and that group names are fresh. WF-SPLIT requires that $e : T$ is in scope, that the conditions $\{C_i\}$ form a disjoint partition of the domain of T , and that there exists exactly one arm labeled by the variant V , extending the path by this branch. WF-SEQ extends Γ with Δ_0 before checking M_1 , implicitly enforcing $\text{dom}(\Delta_0) \cap \text{dom}(\Delta_1) = \emptyset$ via WF-FLD freshness. Finally, WF-LAYOUT checks each variant independently, beginning with an initial environment Γ_0 containing the layout's reference parameters, and requires the terminals to cover all resolved logical fields.

B.4 Build Well-Formedness

In the derivation-free setting, the only possible outcome is copying the fields of the naïve layout, so well-formedness checking verifies there is a surjective mapping onto the physical layout. A *field build* $b ::= \text{build } x$ copies the value of named field x directly to its physical terminal. A *variant build* is $\mathcal{B}_V = \langle b_0, \dots, b_{n-1}; \text{return } e_V \rangle$. A *build specification* $\beta = \{\mathcal{B}_V\}_{V \in \mathcal{V}(\tau)}$ is a set of variant builds, one per variant. The judgment $V; \Delta_V \vdash b \text{ bwf} : \Psi$ checks that a field build is well-formed with respect to terminal bindings Δ_V provided by WF-LAYOUT for V , and records the field set Ψ :

$$\frac{(x, U) \in \mathcal{N}(V) \quad (x, \lfloor U \rfloor_\ell) \in \Delta_V}{\Delta_V \vdash \text{build } x \text{ bwf} : \{x\}} \quad (\text{WF-BFLD})$$

$$\frac{\Delta_V \vdash b_0 \text{ bwf} : \Psi_0 \quad \Delta_V \vdash b_1 \text{ bwf} : \Psi_1 \quad \Psi_0 \cap \Psi_1 = \emptyset}{\Delta_V \vdash b_0; b_1 \text{ bwf} : \Psi_0 \cup \Psi_1} \quad (\text{WF-BSEQ})$$

$$\frac{\forall V \in \mathcal{V}(\tau). \quad \Delta_V \vdash \mathcal{B}_V \text{ bwf} : \Psi_V \quad \{x \mid (x, U) \in \mathcal{F}_\ell(V)\} \subseteq \Psi_V \quad e_V : \mathcal{T}_\ell}{\vdash \beta \text{ bwf}} \quad (\text{WF-BUILD})$$

WF-BFLD requires that x is a logical field of V and that the terminal exists in Δ_V . WF-BSEQ prevents double initialization. Finally, WF-BUILD requires coverage of all fields in $\mathcal{F}_\ell(V)$ and a return expression e_V of type $\mathcal{T}_\ell$.

B.5 Layout Completeness

LEMMA B.2 (FIELD COVERAGE).

If $\Gamma; V \vdash M \text{ wf} : \Delta$, then

$$\{x : T \mid \exists \pi. V \vdash M : (\pi, x : T)\} = \Delta.$$

PROOF. By induction on M .

$M = (x : T)$. Immediate from P-FLD and WF-FLD; both sides equal $\{x : T\}$.

$M = \text{group } g \text{ by } I \{M'\}$. P-GRP prepends $\mathcal{G}(g, I)$ without changing the terminal, and WF-GRP passes Δ unchanged. Apply IH to M' .

$M = \text{split } e \{C_i \rightarrow V_i\{M_i\}\}$. By WF-SPLIT, there is exactly one arm j with $V_j = V$; only M_j is checked and Δ is unchanged. By P-ARM, the same arm contributes terminals. Apply IH to M_j .

$M = M_0; M_1$.

($\subseteq$): Let $x : T$ be reachable in M . By P-SEQ, the derivation comes from M_0 or M_1 . By IH on that member, $x : T \in \Delta_0$ or $x : T \in \Delta_1$, hence $x : T \in \Delta_0 \cup \Delta_1$.

($\supseteq$): Any $x : T \in \Delta_k$ is reachable in M_k by IH, hence reachable in M by P-SEQ. $\square$

LEMMA B.3 (PATH DETERMINISM).

If $\Gamma; V \vdash M \text{ wf} : \Delta$, and $V \vdash M : (\pi_0, x : T)$ and $V \vdash M : (\pi_1, x : T)$, then $\pi_0 = \pi_1$.

PROOF. By induction on M .

$M = (x : T)$. Both paths are ε by P-FLD.

$M = \text{group } g \text{ by } I \{M'\}$. Both paths have prefix $\mathcal{G}(g, I)$ by P-GRP. Apply IH to M' for the suffix.
 $M = \text{split } e \{C_i \rightarrow V_i\{M_i\}\}$. By WF-SPLIT, there is exactly one arm j with $V_j = V$. By P-ARM, both derivations must use arm j , so both paths share prefix $\mathcal{S}(e, V)$. Apply IH to M_j .
 $M = M_0; M_1$. By P-SEQ, each path passes through M_0 or M_1 . If both pass through the same M_k , apply IH to M_k . Otherwise one passes through M_0 and the other through M_1 . By Lemma B.2, $x : T \in \Delta_0$ and $x : T \in \Delta_1$, so $x \in \text{dom}(\Delta_0) \cap \text{dom}(\Delta_1)$. This contradicts $\text{dom}(\Delta_0) \cap \text{dom}(\Delta_1) = \emptyset$, which follows from the freshness condition of WF-FLD propagated through WF-SEQ. $\square$

THEOREM B.4 (LAYOUT COMPLETENESS). *If $\vdash \ell$ wf, then ℓ is complete (Definition B.1).*

PROOF. Let $V \in \mathcal{V}(\tau)$ and $(x, T) \in \mathcal{F}_\ell(V)$. By WF-LAYOUT, $\mathcal{F}_\ell(V) \subseteq \Delta_V$, so $(x : T) \in \Delta_V$. By Lemma B.2, there exists a path $(\pi, x : T)$. Uniqueness of π follows from Lemma B.3. $\square$

B.6 Round-Trip Soundness

For a physical layout ℓ , let

$$\Pi_\ell = \{\pi \mid \exists V, x, T. V \vdash M_\ell : (\pi, x : T)\}$$

be the set of physical paths admitted by the layout body M_ℓ . An *abstract store* is a finite partial map $\sigma : \mathcal{T}_\ell \times \Pi_\ell \rightarrow \text{Val}$. The store is indexed by paths rather than variants, so two variants may share the same location (r, π) for a common field; injectivity of $\mathcal{T}_\ell$ ensures that cells $(r, _)$ and $(r', _)$ are disjoint whenever $r \neq r'$. We define

$$\text{enc}_\beta : \tau \rightarrow (\Sigma \times \mathcal{T}_\ell) \quad \text{dec}_\ell : \Sigma \times \mathcal{T}_\ell \rightarrow \tau$$

informally, where Σ denotes the set of all abstract stores. A full operational semantics for both is left for future work. Instead, we sketch why the structural argument below is sound under the well-formedness conditions above.

THEOREM B.5 (ROUND-TRIP SOUNDNESS). *If $\vdash \ell$ wf and $\vdash \beta$ bwf, then $\text{dec}_\ell(\text{enc}_\beta(t)) = t$.*

PROOF SKETCH. By induction on $t = V(\overline{v_x})$ with $\mathcal{N}(V) = \{(x_i, T_i)\}_{i=1}^k$. The sketch assumes that enc_β allocates a fresh reference r , writes each field to its unique terminal path (as guaranteed by Theorem B.4), and that dec_ℓ is a syntax-directed traversal reading values back from those same paths. By WF-BUILD, variant V has a unique build procedure $\mathcal{B}_V$ covering all names in $\mathcal{F}_\ell(V)$; by WF-BSEQ, each name is initialized exactly once. By Theorem B.4, every field in $\mathcal{N}(V)$ has exactly one terminal path. For each $(x_i, T_i) \in \mathcal{N}(V)$:

$T_i \in T \setminus \{\alpha\}$. By WF-BFLD, the encoder writes v_{x_i} to the unique terminal $x_i : T_i \in \Delta_V$. The decoder follows path π_i and reads v_{x_i} directly.

$T_i = \alpha$. The encoder recursively computes $(\sigma_i, r_i) = \text{enc}_\beta(v_{x_i})$, writes $r_i : \mathcal{T}_\ell$ to terminal $x_i : \mathcal{T}_\ell \in \Delta_V$, and accumulates σ_i by disjoint union. The decoder follows path π_i , reads r_i , and applies IH to obtain $\text{dec}_\ell(\sigma_i, r_i) = v_{x_i}$.

Since every field is recovered, $\text{dec}_\ell(\text{enc}_\beta(t)) = V(\overline{v_x}) = t$. $\square$

C Design Space Exploration: Dataset & Analysis Script

For the layout design space exploration in Section 9.2, we provide the complete dataset and analysis script in the Supplemental Material. The script requires Python 3 with numpy, matplotlib, and pandas installed. CSV column descriptions and example usage are provided in the script.

D Scion Layout & Build Specifications

D.1 Layout & Build Specifications for sg-eq

```

1 type BVH(low : f32x3, high : f32x3) = Interior(left : BVH, right : BVH) | Leaf(nprims: u4, data : Triangle[nprims]);
2
3 func dequantize(v: u30, bins: f32x3) -> f32x3 {
4   let x_: u32 = (v >> 20) & 1023; let y_: u32 = (v >> 10) & 1023; let z_: u32 = (v >> 0) & 1023;
5   return f32x3 { fmul_rd(x_ as f32, bins.x), fmul_rd(y_ as f32, bins.y), fmul_rd(z_ as f32, bins.z) };
6 }
7 func construct_bins_inverse(low: f32x3, high: f32x3) -> f32x3 {
8   let L1: f32x3 = f32x3{fsub_ru(high.x, low.x), fsub_ru(high.y, low.y), fsub_ru(high.z, low.z)};
9   let L2: mut f32x3 = {1.0, 1.0, 1.0};
10  if (L1.x > 0.0) { L2.x = L1.x; } if (L1.y > 0.0) { L2.y = L1.y; } if (L1.z > 0.0) { L2.z = L1.z; }
11  return f32x3{fdiv_rd(1023.0, L2.x), fdiv_rd(1023.0, L2.y), fdiv_rd(1023.0, L2.z)};
12 }
13 func construct_bins(low: f32x3, high: f32x3) -> f32x3 {
14   let bins_inverse: f32x3 = construct_bins_inverse(low, high);
15   return f32x3 {frcp_rd(bins_inverse.x), frcp_rd(bins_inverse.y), frcp_rd(bins_inverse.z)};
16 }
17 func quantize_lo(current: f32x3, world: f32x3, bin_inverse: f32x3) -> u32 {
18   let x: u32 = floorf(fmul_rd(fsub_rd(current.x, world.x), bin_inverse.x)) as u32;
19   let y: u32 = floorf(fmul_rd(fsub_rd(current.y, world.y), bin_inverse.y)) as u32;
20   let z: u32 = floorf(fmul_rd(fsub_rd(current.z, world.z), bin_inverse.z)) as u32;
21   return ((x & 1023u) << 20u) | ((y & 1023u) << 10u) | (z & 1023u);
22 }
23 func quantize_hi(current: f32x3, world: f32x3, bin_inverse: f32x3) -> u32 {
24   let x: u32 = floorf(fmul_rd(fsub_rd(world.x, current.x), bin_inverse.x)) as u32;
25   let y: u32 = floorf(fmul_rd(fsub_rd(world.y, current.y), bin_inverse.y)) as u32;
26   let z: u32 = floorf(fmul_rd(fsub_rd(world.z, current.z), bin_inverse.z)) as u32;
27   return ((x & 1023u) << 20u) | ((y & 1023u) << 10u) | (z & 1023u);
28 }
29 // Snapped-grid extent quantization using 2^10-1 bins.
30 layout BVH(index: u32) {
31   primitive_count : u32;
32   primitives : Triangle[primitive_count];
33   wlow: f32x3; whigh: f32x3;
34   bins: f32x3; bins_inv: f32x3;
35   node_count : u32;
36   group nodes[size=node_count] by index {
37     q_min: u30; q_max: u30; nprims: u4;
38     low = fadd_rd(wlow, dequantize(q_min, bins));
39     high = fsub_ru(whigh, dequantize(q_max, bins));
40     split nprims {
41       0 -> Interior {
42         offset : u32; left = index + 1; right = index + offset;
43       };
44       > 0 -> Leaf {
45         poffset : u32; data = primitives[poffset : poffset + nprims];
46       };
47     };
48   };
49 };
50 build BVH[order=pre] {
51   build Interior(low: f32x3, high: f32x3, left: BVH, right: BVH) {
52     build root {
53       build wlow = low;
54       build whigh = high;
55       build bins_inv = construct_bins_inverse(low, high);
56       build bins = construct_bins(low, high);
57     };
58     build q_min = quantize_lo(low, wlow, bins_inv);
59     build q_max = quantize_hi(high, whigh, bins_inv);
60     build nprims = 0;
61     build left;
62     let R: u32 = build right;
63     build offset = R - this;
64     return this;
65   };
66 };
67 build Leaf(low: f32x3, high: f32x3, nprims: u4, data: Triangle[nprims]) {
68   build q_min = quantize_lo(low, wlow, bins_inv);
69   build q_max = quantize_hi(high, whigh, bins_inv);
70   build nprims;
71   build poffset = append(data, nprims);
72   return this;
73 };
74 };

```


D.2 Layout & Build Specifications for bv8-q8-ci

```

1 type BVH = Interior(children: BVH[8], lo: f32x3x8, hi: f32x3x8) | Leaf(nprims: u8, data: Triangle[nprims]);
2 type qbox3(lo: u8x3, hi: u8x3);
3
4 func dequantize_bounds_lo(mlo: f32x3, mex: f32x3, bound: qbox3x8) -> f32x3x8 {
5   let rcp: f32 = 1 / 255.0;
6   return f32x3x8 {
7     mlo + (bound[0].lo as f32x3 * rcp) * mex, mlo + (bound[1].lo as f32x3 * rcp) * mex,
8     mlo + (bound[2].lo as f32x3 * rcp) * mex, mlo + (bound[3].lo as f32x3 * rcp) * mex,
9     mlo + (bound[4].lo as f32x3 * rcp) * mex, mlo + (bound[5].lo as f32x3 * rcp) * mex,
10    mlo + (bound[6].lo as f32x3 * rcp) * mex, mlo + (bound[7].lo as f32x3 * rcp) * mex
11  };
12 }
13 func dequantize_bounds_hi(mlo: f32x3, mex: f32x3, bound: qbox3x8) -> f32x3x8 {
14   let rcp: f32 = 1 / 255.0;
15   return f32x3x8 {
16     mlo + (bound[0].hi as f32x3 * rcp) * mex, mlo + (bound[1].hi as f32x3 * rcp) * mex,
17     mlo + (bound[2].hi as f32x3 * rcp) * mex, mlo + (bound[3].hi as f32x3 * rcp) * mex,
18     mlo + (bound[4].hi as f32x3 * rcp) * mex, mlo + (bound[5].hi as f32x3 * rcp) * mex,
19     mlo + (bound[6].hi as f32x3 * rcp) * mex, mlo + (bound[7].hi as f32x3 * rcp) * mex
20  };
21 }
22 func tfloor(f: f32x3) -> u8x3 { let f1: f32x3 = floorf(f); let f2: f32x3 = max(0.0, min(f1, 255.0)); return f2 as u8x3; }
23 func tceil(f: f32x3) -> u8x3 { let f1: f32x3 = ceilf(f); let f2: f32x3 = max(0.0, min(f1, 255.0)); return f2 as u8x3; }
24 // Use 32-bit reference type; this still allows for 2^25 primitives.
25 // The first Interior node will have value 1, thus we provide this as the default.
26 layout BVH(I: u32 = 1u) {
27   primitive_count: u32; primitives : Triangle[primitive_count]; interior_count: u32;
28   indirect_group Interiors[size=interior_count] {
29     mlo: f32x3; mex: f32x3; child_bounds: qbox3x8; children: u32x8;
30     lo = dequantize_bounds_lo(mlo, mex, child_bounds); hi = dequantize_bounds_hi(mlo, mex, child_bounds);
31   };
32   group by I {
33     split I[0:1] {
34       1 -> Interior from Interiors[I[2:31]];
35       0 -> Leaf { let O: u32 = I[7:31]; nprims = (I[2:6] + 1) as u8; data = primitives[0 : O + nprims]; };
36     };
37   };
38 };
39
40 func quantize_bounds(low: f32x3x8, high: f32x3x8) -> qbox3x8 {
41   let mlo: f32x3 = compute_merged_low(low); let mex: f32x3 = compute_merged_extent(low, high);
42   let rcp: f32x3 = (1.0 / mex) * 255.0;
43   return qbox3x8 {
44     qbox3 {tfloor((low[0] - mlo) * rcp), tceil((high[0] - mlo) * rcp)},
45     qbox3 {tfloor((low[1] - mlo) * rcp), tceil((high[1] - mlo) * rcp)},
46     qbox3 {tfloor((low[2] - mlo) * rcp), tceil((high[2] - mlo) * rcp)},
47     qbox3 {tfloor((low[3] - mlo) * rcp), tceil((high[3] - mlo) * rcp)},
48     qbox3 {tfloor((low[4] - mlo) * rcp), tceil((high[4] - mlo) * rcp)},
49     qbox3 {tfloor((low[5] - mlo) * rcp), tceil((high[5] - mlo) * rcp)},
50     qbox3 {tfloor((low[6] - mlo) * rcp), tceil((high[6] - mlo) * rcp)},
51     qbox3 {tfloor((low[7] - mlo) * rcp), tceil((high[7] - mlo) * rcp)}
52   };
53 }
54 func compute_merged_low(low: f32x3x8) -> f32x3 {
55   return min(low[0], min(low[1], min(low[2], min(low[3],
56     min(low[4], min(low[5], min(low[6], low[7])))))));
57 }
58 func compute_merged_extent(lo: f32x3x8, hi: f32x3x8) -> f32x3 {
59   let mlo: f32x3 = min(lo[0], min(lo[1], min(lo[2], min(lo[3],
60     min(lo[4], min(lo[5], min(lo[6], lo[7])))))));
61   let mhi: f32x3 = max(hi[0], max(hi[1], max(hi[2], max(hi[3],
62     max(hi[4], max(hi[5], max(hi[6], hi[7])))))));
63   return mhi - mlo;
64 }
65 build BVH[order=pre] {
66   build Interior(children: BVH[8], lo: f32x3x8, hi: f32x3x8) {
67     build mlo = compute_merged_low(lo); build mex = compute_merged_extent(lo, hi);
68     build child_bounds = quantize_bounds(lo, hi);
69     build children;
70     return ((this << 2u) | 1u) as u32; // `1` tag for Interior.
71   };
72   build Leaf(nprims: u8, data: Triangle[nprims]) {
73     let poffset: u32 = append(data, nprims);
74     return ((poffset << 7u) | ((nprims - 1u) << 2u) | 0u) as u32; // `0` tag for Leaf.
75   };
76 };

```

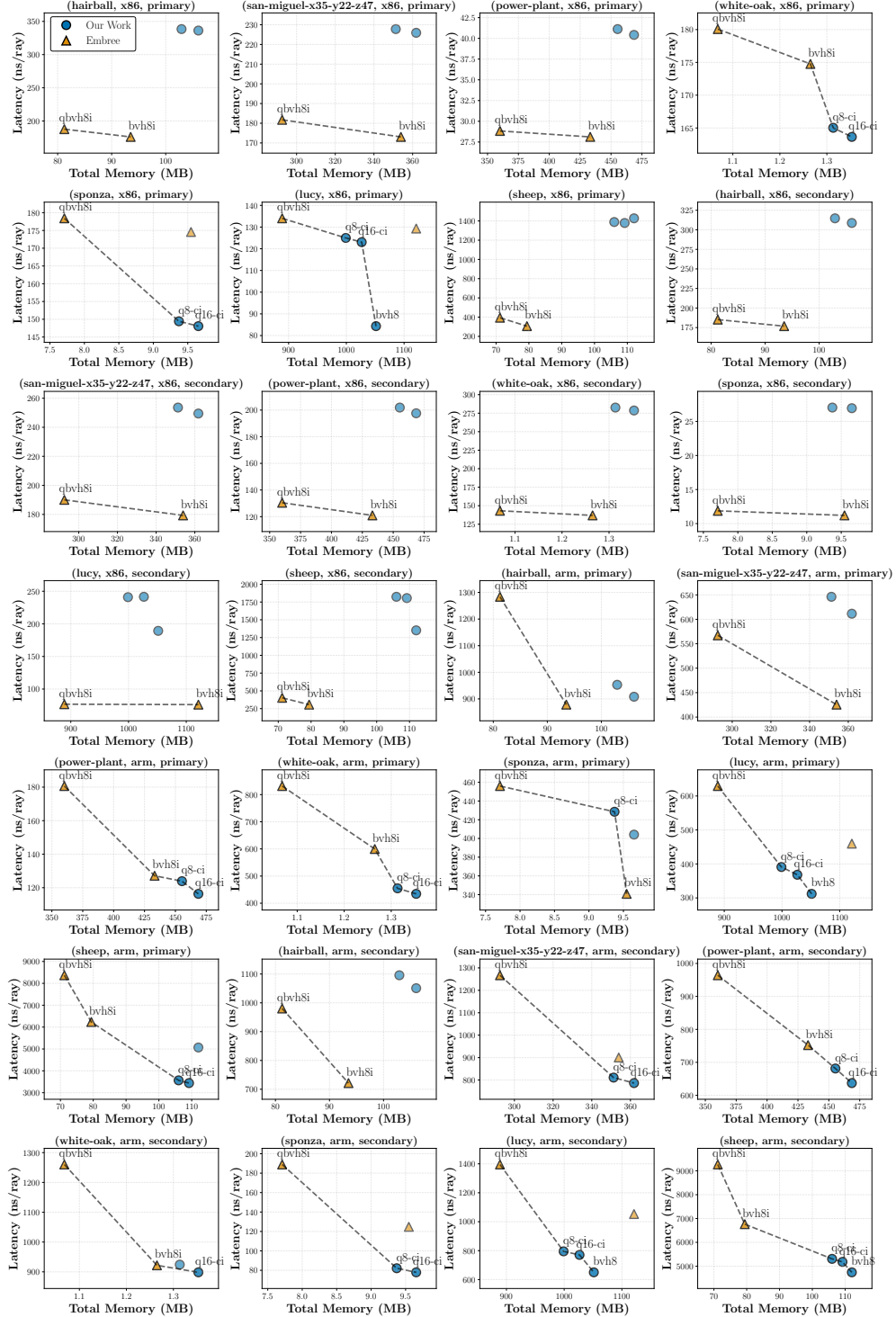
D.3 Layout & Build Specifications for pbrt-q16

```

1 type BVH(low : f32x3, high : f32x3) = Interior(left : BVH, right : BVH) | Leaf(nprims: u4, data : Triangle[nprims]);
2 type q16x3(lo: u16x3, hi: u16x3);
3
4 func dequantize_lo(mlo: f32x3, mex: f32x3, bound: q16x3) -> f32x3 {
5   let rcp: f32 = 1.0 / 65535.0;
6   return mlo + ((bound.lo as f32x3) * rcp) * mex;
7 }
8 func dequantize_hi(mlo: f32x3, mex: f32x3, bound: q16x3) -> f32x3 {
9   let rcp: f32 = 1.0 / 65535.0;
10  return mlo + ((bound.hi as f32x3) * rcp) * mex;
11 }
12 func vu_floor(f: f32x3) -> u16x3 {
13   let f1: f32x3 = floorf(f);
14   let f2: f32x3 = max(0.0, min(f1, 65535.0));
15   return f2 as u16x3;
16 }
17 func vu_ceil(f: f32x3) -> u16x3 {
18   let f1: f32x3 = ceilf(f);
19   let f2: f32x3 = max(0.0, min(f1, 65535.0));
20   return f2 as u16x3;
21 }
22 // PBRT implicit indexing and bit stealing with snapped-grid extent quantization using 2^16-1 bins.
23 // Additionally, this uses the AABB quantization scheme described in the Compressed-Leaf BVH work.
24 layout BVH(index: u32) {
25   primitive_count : u32;
26   primitives : Triangle[primitive_count];
27   world_low : f32x3; world_extnt : f32x3;
28   node_count : u32;
29   group nodes[size=node_count, align=16] by index {
30     bounds_q : q16x3;
31     low = dequantize_lo(world_low, world_extnt, bounds_q);
32     high = dequantize_hi(world_low, world_extnt, bounds_q);
33     nprims: u4;
34     split nprims {
35       0 -> Interior {
36         c_offset: u28;
37         left = index + 1;
38         right = index + c_offset;
39       };
40       > 0 -> Leaf {
41         p_offset: u28;
42         data = primitives[p_offset : p_offset + nprims];
43       };
44     };
45   };
46 };
47 func quantize_bounds(low: f32x3, high: f32x3, mlo: f32x3, mex: f32x3) -> q16x3 {
48   let rcp: f32x3 = (1.0 / mex) * 65535.0;
49   return q16x3 { vu_floor((low - mlo) * rcp), vu_ceil((high - mlo) * rcp) };
50 }
51 build BVH[order=pre] {
52   build Interior(low: f32x3, high: f32x3, left: BVH, right: BVH) {
53     build root {
54       build world_low = low;
55       build world_extnt = high - low;
56     };
57     build bounds_q = quantize_bounds(low, high, world_low, world_extnt);
58     build left;
59     let R: u32 = build right;
60     build c_offset = R - this;
61     return this;
62   };
63   build Leaf(low: f32x3, high: f32x3, nprims: u4, data: Triangle[nprims]) {
64     build bounds_q = quantize_bounds(low, high, world_low, world_extnt);
65     build p_offset = append(data, nprims);
66     build nprims;
67     return this;
68   };
69 };
70 };

```

E Closest Hit Ray Tracing Pareto Frontier (Embree vs Scienc)



F Generated C++ Code for Closest Hit Ray Tracing with PBRTv4

```

1 // Logical Tree
2 struct Interior;
3 struct Leaf;
4 using BVH = std::variant<Interior, Leaf>;
5 struct Interior { float3 low; float3 high; BVH* left; BVH* right; };
6 struct AABB { float3 low; float3 high; };
7 struct Leaf { float3 low; float3 high; uint16_t nprims; Triangle* data; };
8
9 struct Ray { float3 origin; float3 direction; float tmax = INF; };
10 struct Triangle { float3 p0; float3 p1; float3 p2; };
11
12 // Physical Tree
13 struct InteriorArm {
14     uint32_t c_o;
15 } __attribute__((packed));
16 struct LeafArm {
17     uint32_t p_o;
18 } __attribute__((packed));
19 struct alignas(32) Nodes {
20     float3 low; float3 high; uint16_t nprims;
21     uint8_t S[4]; // p_o or c_o
22 } __attribute__((packed));
23 struct LinearBVH {
24     uint32_t P; uint32_t N; Triangle* primitives; Nodes* nodes;
25 } __attribute__((packed));
26
27 // Concretization, generated from the Tree Traversal and Layout Language.
28 void closest_hit(
29     const uint32_t I,
30     const LinearBVH* __restrict__ PT,
31     const Ray* __restrict__ ray,
32     std::tuple<float, Triangle* __restrict__ __best0
33 ) {
34     if ((*PT).nodes[I].nprims == 0u) {
35         if (intersects_Ray_AABB(
36             ray, &AABB{
37                 .low = (*PT).nodes[I].low,
38                 .high = (*PT).nodes[I].high
39             }) &&
40             distmin_Ray_AABB(
41                 ray, &AABB{
42                     .low = (*PT).nodes[I].low,
43                     .high = (*PT).nodes[I].high
44                 }) < std::get<0>(* __best0)) {
45                 closest_hit(I + 1u, PT, ray, __best0);
46                 closest_hit(
47                     I + reinterpret<InteriorArm>((*PT).nodes[index].S).c_o,
48                     PT, ray, __best0
49                 );
50             }
51         } else {
52             for (uint32_t j =
53                 reinterpret<LeafArm>((*PT).nodes[I].S).p_o;
54                 j <
55                 (reinterpret<LeafArm>((*PT).nodes[I].S).p_o +
56                 (*PT).nodes[I].nprims); ++j) {
57                 if (intersects_Ray_AABB(
58                     ray, &AABB{
59                         .low = (*PT).nodes[I].low,
60                         .high = (*PT).nodes[I].high
61                     }) &&
62                     distmin_Ray_AABB(
63                         ray, &AABB{
64                             .low = (*PT).nodes[I].low,
65                             .high = (*PT).nodes[I].high
66                         }) < std::get<0>(* __best0)) {
67                         if (intersects_Ray_Triangle(ray, &(*PT).primitives[j])
68                             && distmin_Ray_Triangle(ray, &(*PT).primitives[j])
69                             < std::get<0>(* __best0)) {
70                             (* __best0) = {
71                                 distmin_Ray_Triangle(ray, &(*PT).primitives[j]),
72                                 (*PT).primitives[j]
73                             };
74                         }
75                     }
76                 }
77             }
78         return;
79     }
}

1 // Recursive build construction. Every variant has its own procedure.
2 uint32_t rec_build(
3     const BVH* __restrict__ node,
4     LinearBVH* __restrict__ PT,
5     size_t* __restrict__ nodes_index,
6     size_t* __restrict__ primitives_index
7 ) {
8     return std::visit(overloaded{
9         [&](const Interior& node) {
10             const size_t this_index = (*nodes_index);
11             (*nodes_index) += 1u;
12             (*PT).nodes[this_index].low = node.low;
13             (*PT).nodes[this_index].high = node.high;
14             (*PT).nodes[this_index].nprims = 0;
15             rec_build(node.left, PT, nodes_index, primitives_index);
16             const uint32_t right_index =
17                 rec_build(node.right, PT, nodes_index, primitives_index);
18             reinterpret_cast<InteriorArm*>(&(*PT).nodes[this_index].S
19             )->c_o = right_index - this_index;
20             return this_index;
21         },
22         [&](const Leaf& node) {
23             const size_t this_index = (*nodes_index);
24             (*nodes_index) += 1u;
25             (*PT).nodes[this_index].low = node.low;
26             (*PT).nodes[this_index].high = node.high;
27             (*PT).nodes[this_index].nprims = node.nprims;
28             reinterpret_cast<LeafArm*>(&(*PT).nodes[this_index].S
29             )->p_o = (*primitives_index);
30             for (uint16_t __p = 0u; __p < node.nprims; ++__p) {
31                 (*PT).primitives[__p + (*primitives_index)] = node.data[__p];
32             }
33             (*primitives_index) += node.nprims;
34             return this_index;
35         }
36     }, *node);
37 }
38
39 // Recursive count procedure for allocation. The compiler can infer
40 // which groups should be incremented by referring to the layout.
41 void rec_count(
42     const BVH* __restrict__ node,
43     LinearBVH* __restrict__ PT
44 ) {
45     return std::visit(overloaded{
46         [&](const Interior& node) {
47             rec_count(node.left, PT);
48             rec_count(node.right, PT);
49             (*PT).N += 1u;
50         },
51         [&](const Leaf& node) {
52             (*PT).P += node.nprims;
53             (*PT).N += 1u;
54         }
55     }, *node);
56 }
57
58 // Constructor, generated from the Build Language.
59 LinearBVH build(const BVH* __restrict__ LT) {
60     LinearBVH PT;
61     size_t primitives_index = 0u;
62     size_t nodes_index = 0u;
63     PT.P = 0u;
64     PT.N = 0u;
65     rec_count(LT, (&PT));
66     Triangle* primitives = reinterpret_cast<Triangle*>(&
67         malloc(sizeof(Triangle) * PT.P)
68     );
69     PT.primitives = primitives;
70     Nodes* nodes = reinterpret_cast<Nodes*>(&
71         std::aligned_alloc(32, ((sizeof(Nodes) * PT.N) + 31) / 32) * 32
72     );
73     PT.nodes = nodes;
74     rec_build(LT, (&PT), (&nodes_index), (&primitives_index));
75     return PT;
76 }

```

Fig. 20. C++ code generated by SCION for Closest Hit Ray Tracing with the PBRTv4 layout. We provide the unoptimized version because renaming occurs during the optimization passes, resulting in inscrutable code.

G SCION-Implemented Tree Traversal Algorithms

Closest Hit Ray Tracing

```

1 type Ray(origin: f32x3, direction: f32x3, tmax: f32 = ∞);
2 type FInterval(low: f32, high: f32);
3 type TriangleIntersection(b0: f32, b1: f32, b2: f32, t: f32);
4 type AABB(low: f32x3, high: f32x3);
5 type Triangle(p0: f32x3, p1: f32x3, p2: f32x3);
6 type BVH(low: f32x3, high: f32x3)
7   = Interior(left: BVH, right: BVH)
8   | Leaf(nprims: u8, data: Triangle[nprims]);
9 // Ray-AABB intersection following Embree's approach.
10 func intersectsp_ray_aabb(r: Ray, b: AABB) -> option[FInterval] {
11   let rdir: f32x3 = 1.0 / r.direction; let is_n: boolx3 = r.direction < 0.0;
12   let nx: f32 = select(is_n.x, b.high.x, b.low.x); let fx: f32 = select(is_n.x, b.low.x, b.high.x);
13   let ny: f32 = select(is_n.y, b.high.y, b.low.y); let fy: f32 = select(is_n.y, b.low.y, b.high.y);
14   let nz: f32 = select(is_n.z, b.high.z, b.low.z); let fz: f32 = select(is_n.z, b.low.z, b.high.z);
15   let t_nx: f32 = (nx - r.origin.x) * rdir.x; let t_fx: f32 = (fx - r.origin.x) * rdir.x;
16   let t_ny: f32 = (ny - r.origin.y) * rdir.y; let t_fy: f32 = (fy - r.origin.y) * rdir.y;
17   let t_nz: f32 = (nz - r.origin.z) * rdir.z; let t_fz: f32 = (fz - r.origin.z) * rdir.z;
18   let t_near: f32 = max(0.0, max(t_nx, max(t_ny, t_nz))); let t_far: f32 = min(r.tmax, min(t_fx, min(t_fy, t_fz)));
19   if t_near <= t_far { return FInterval(t_near, t_far); } return {};
20 }
21 // Moeller-Trumbore ray-triangle intersection following Embree's approach.
22 func intersectsp_ray_tri_mt(ray: Ray, tri: Triangle) -> option[TriangleIntersection] {
23   let e1: f32x3 = tri.p0 - tri.p1; let e2: f32x3 = tri.p2 - tri.p0; let ng: f32x3 = cross(e2, e1);
24   let c: f32x3 = tri.p0 - ray.origin; let r: f32x3 = cross(c, ray.direction); let D: f32 = dot(ng, ray.direction);
25   if D == 0.0 { return {}; }
26   let abs_D: f32 = abs(D); let sgn_D: u32 = (D > 0) & 2147483648u;
27   let u_raw: f32 = ((dot(r, e2) to u32) ^ sgn_D) to f32; let v_raw: f32 = ((dot(r, e1) to u32) ^ sgn_D) to f32;
28   if !(u_raw >= 0.0 && v_raw >= 0.0 && u_raw + v_raw <= abs_D) { return {}; }
29   let t_raw: f32 = (((dot(ng, c) ^ sgn_D) to u32) to f32);
30   if !(abs_D * 0.0 < t_raw && t_raw <= abs_D * ray.tmax) { return {}; }
31   let inv_abs_D: f32 = 1.0 / abs_D;
32   let t: f32 = t_raw * inv_abs_D; let u: f32 = u_raw * inv_abs_D; let v: f32 = v_raw * inv_abs_D;
33   let b0: f32 = 1.0 - u - v; let b1: f32 = u; let b2: f32 = v;
34   return TriangleIntersection(b0, b1, b2, t);
35 }
36 // Pluecker coordinates ray-triangle intersection following Embree's approach.
37 func intersectsp_ray_tri_pc(ray: Ray, tri: Triangle) -> option[TriangleIntersection] {
38   let v0: f32x3 = tri.p0 - ray.origin; let v1: f32x3 = tri.p1 - ray.origin; let v2: f32x3 = tri.p2 - ray.origin;
39   let e0: f32x3 = v2 - v0; let e1: f32x3 = v0 - v1; let e2: f32x3 = v1 - v2;
40   let u_raw: f32 = dot(cross(e0, v2 + v0), ray.direction);
41   let v_raw: f32 = dot(cross(e1, v0 + v1), ray.direction);
42   let w_raw: f32 = dot(cross(e2, v1 + v2), ray.direction);
43   let uvw: f32 = u_raw + v_raw + w_raw; let e: f32 = e * abs(uvw);
44   let min_uvw: f32 = min({u_raw, v_raw, w_raw}); let max_uvw: f32 = max({u_raw, v_raw, w_raw});
45   if !(min_uvw >= -e || max_uvw <= e) { return {}; }
46   let ng: f32x3 = cross(e0, e1); let den: f32 = 2.0 * dot(ng, ray.direction); let t_raw: f32 = 2.0 * dot(v0, ng);
47   let t: f32 = t_raw / den; if !(t >= 0.0 && t <= ray.tmax) { return {}; } if den == 0.0 { return {}; }
48   let inv_uvw: f32 = 1.0 / uvw; let b0: f32 = w_raw * inv_uvw; let b1: f32 = u_raw * inv_uvw; let b2: f32 = v_raw * inv_uvw;
49   if b0 < 0.0 || b1 < 0.0 || b2 < 0.0 { return {}; } return TriangleIntersection(b0, b1, b2, t);
50 }
51 func intersects(r: Ray, b: AABB) -> bool {
52   let I: option[FInterval] = intersectsp_ray_aabb(r, b); if I { return I.low < r.tmax && I.high > 0; } return false;
53 }
54 func distmin(r: Ray, b: AABB) -> f32 {
55   let interval: option[FInterval] = intersectsp_ray_aabb(r, b); if interval { return interval.low; } return ∞;
56 }
57 func distmin(ray: Ray, tri: Triangle) -> f32 {
58   // *NOTE*: chosen ray-triangle intersection method is specified in the Evaluation.
59   let I: option[TriangleIntersection] = intersectsp_ray_tri_mt(ray, tri); if I { return I.t; } return ∞;
60 }
61 func closest_hit(ray: Ray, bvh: BVH, best: mut (f32, Triangle)) =
62   match bvh {
63   | Interior(low, high, left, right) ->
64     if intersects(ray, AABB(low, high)) && (distmin(ray, AABB(low, high)) < best[0]) {
65       closest_hit(ray, left, best);
66       closest_hit(ray, right, best);
67     }
68   | Leaf(low, high, nprims, data) ->
69     if intersects(ray, AABB(low, high)) {
70       foreach t in data {
71         if intersects(ray, t) && distmin(ray, t) < best[0] {
72           best = (distmin(ray, t), t);
73         }
74       }
75     }
76 }

```

Closest Point Query

```

1 type AABB(low: f32x3, high: f32x3);
2 type Point(v: f32x3);
3 type Triangle(p0: f32x3, p1: f32x3, p2: f32x3);
4 type BVH(low: f32x3, high: f32x3)
5   = Interior(left: BVH, right: BVH)
6   | Leaf(nprims: u16, data: Triangle[nprims]);
7
8 // Returns the closest point and barycentric coordinates.
9 // Ref: Real-Time Collision Detection [Ericson et al.] Ch. 5.1.5
10 func distmin_point_triangle(pt: Point, tri: Triangle) -> (Point, Point) {
11   let p: f32x3 = pt.v;
12   let a: f32x3 = tri.p0; let b: f32x3 = tri.p1; let c: f32x3 = tri.p2;
13   let ab: f32x3 = b - a; let ac: f32x3 = c - a; let ap: f32x3 = p - a;
14   let d1: f32 = dot(ab, ap); let d2: f32 = dot(ac, ap);
15   if d1 <= 0.0 && d2 <= 0.0 { return (a, {{1.0, 0.0, 0.0}}); }
16   let bp: f32x3 = p - b; let d3: f32 = dot(ab, bp); let d4: f32 = dot(ac, bp);
17   if d3 >= 0.0 && d4 <= d3 { return (b, {{0.0, 1.0, 0.0}}); }
18   let vc: f32 = d1*d4 - d3*d2;
19   if vc <= 0.0 && d1 >= 0.0 && d3 <= 0.0 {
20     let v0: f32 = d1 / (d1 - d3); return (a + v0 * ab, {{1.0 - v0, v0, 0.0}});
21   }
22   let cp: f32x3 = p - c; let d5: f32 = dot(ab, cp); let d6: f32 = dot(ac, cp);
23   if d6 >= 0.0 && d5 <= d6 { return (c, {{0.0, 0.0, 1.0}}); }
24   let vb: f32 = d5*d2 - d1*d6;
25   if vb <= 0.0 && d2 >= 0.0 && d6 <= 0.0 {
26     let w0: f32 = d2 / (d2 - d6); return (a + w0 * ac, {{1.0-w0, 0.0, w0}});
27   }
28   let va: f32 = d3*d6 - d5*d4;
29   if va <= 0.0 && (d4 - d3) >= 0.0 && (d5 - d6) >= 0.0 {
30     let w1: f32 = (d4 - d3) / ((d4 - d3) + (d5 - d6)); return (b + w1 * (c - b), {{0.0, 1.0-w1, w1}});
31   }
32   let D: f32 = 1.0 / (va + vb + vc); let v: f32 = vb * D; let w: f32 = vc * D; let u: f32 = va * D;
33   return (a + ab * v + ac * w, {{u, v, w}});
34 }
35 func distmin(p: Point, tri: Triangle) -> f32 {
36   let ps: (Point, Point) = distmin_point_triangle(p, tri); let x: f32x3 = p.v - ps[0].v; return dot(x, x);
37 }
38 // Ref: Real-Time Collision Detection [Ericson et al.] Ch. 5.1.3.1
39 func square_distance_point_aabb(pt: Point, a: AABB) -> f32 {
40   let v: f32x3 = pt.v;
41   let sq_low: f32x3 = (a.low - v) * (a.low - v); let low: f32x3 = select(v < a.low, sq_low, 0.0);
42   let sq_high: f32x3 = (v - a.high) * (v - a.high); let high: f32x3 = select(v > a.high, sq_high, 0.0);
43   return sum(low + high);
44 }
45 func distmin(pt: Point, a: AABB) -> f32 {
46   return square_distance_point_aabb(pt, a);
47 }
48 func distmax(pt: Point, a: AABB) -> f32 {
49   let u: f32x3 = a.low - pt.v; let v: f32x3 = pt.v - a.high; let d: f32x3 = min(u, v); return dot(d, d);
50 }
51 func closest_point(p: Point, bvh: BVH, best: mut (f32, Point)) =
52   match bvh {
53   | Interior(low, high, left, right) ->
54     if distmin(p, AABB(low, high)) < best[0] {
55       let upper_bound: f32 = distmax(p, AABB(low, high));
56       if upper_bound < best[0] { best = (upper_bound, best[1]); }
57       // Children sorting optimization.
58       let La: AABB = match left {
59       | Interior(l1, h1, _, _) -> AABB(l1, h1)
60       | Leaf(l1, h1, _, _) -> AABB(l1, h1)
61       }
62       let Ra: AABB = match right {
63       | Interior(lr, hr, _, _) -> AABB(lr, hr)
64       | Leaf(lr, hr, _, _) -> AABB(lr, hr)
65       }
66       let L: f32 = distmin(p, La); let R: f32 = distmin(p, Ra);
67       if L < R {
68         closest_triangle(p, left, best); closest_triangle(p, right, best);
69       } else {
70         closest_triangle(p, right, best); closest_triangle(p, left, best);
71       }
72     }
73   | Leaf(low, high, nprims, data) ->
74     if distmin(p, AABB(low, high)) < best[0] {
75       foreach t in data {
76         if distmin(p, t) < best[0] { best = (distmin(p, t), t); }
77       }
78     }
79 }

```


Collision Detection

```

1 type AABB(low: f32x3, high: f32x3);
2 type Triangle(p0: f32x3, p1: f32x3, p2: f32x3);
3 type BVH(low: f32x3, high: f32x3)
4   = Interior(left: BVH, right: BVH)
5   | Leaf(nprims: u16, data: Triangle[nprims]);
6
7 func project6(ax: f32x3, p1: f32x3, p2: f32x3, p3: f32x3,
8   q1: f32x3, q2: f32x3, q3: f32x3) -> i32 {
9   let P1: f32 = dot(ax, p1); let P2: f32 = dot(ax, p2); let P3: f32 = dot(ax, p3);
10  let Q1: f32 = dot(ax, q1); let Q2: f32 = dot(ax, q2); let Q3: f32 = dot(ax, q3);
11  let mn1: f32 = min(min(P1, P2), P3); let mx2: f32 = max(max(Q1, Q2), Q3); if mn1 > mx2 { return 0; }
12  let mx1: f32 = max(max(P1, P2), P3); let mn2: f32 = min(min(Q1, Q2), Q3); if mn2 > mx1 { return 0; }
13  return 1;
14 }
15 // SAT triangle-triangle intersection following FCL's approach.
16 func SAT_triangle_intersection(
17   P1: f32x3, P2: f32x3, P3: f32x3, Q1: f32x3, Q2: f32x3, Q3: f32x3
18 ) -> bool {
19   let p1: f32x3 = {0.0, 0.0, 0.0}; let p2: f32x3 = P2 - P1; let p3: f32x3 = P3 - P1;
20   let q1: f32x3 = Q1 - P1; let q2: f32x3 = Q2 - P1; let q3: f32x3 = Q3 - P1;
21   let e1: f32x3 = p2 - p1; let e2: f32x3 = p3 - p2; let n1: f32x3 = cross(e1, e2);
22   if project6(n1, p1, p2, p3, q1, q2, q3) == 0 { return false; }
23   let f1: f32x3 = q2 - q1; let f2: f32x3 = q3 - q2; let m1: f32x3 = cross(f1, f2);
24   if project6(m1, p1, p2, p3, q1, q2, q3) == 0 { return false; }
25   let ef11: f32x3 = cross(e1, f1); if project6(ef11, p1, p2, p3, q1, q2, q3) == 0 { return false; }
26   let ef12: f32x3 = cross(e1, f2); if project6(ef12, p1, p2, p3, q1, q2, q3) == 0 { return false; }
27   let f3: f32x3 = q1 - q3; let ef13: f32x3 = cross(e1, f3); if project6(ef13, p1, p2, p3, q1, q2, q3) == 0 { return false; }
28   let ef21: f32x3 = cross(e2, f1); if project6(ef21, p1, p2, p3, q1, q2, q3) == 0 { return false; }
29   let ef22: f32x3 = cross(e2, f2); if project6(ef22, p1, p2, p3, q1, q2, q3) == 0 { return false; }
30   let ef23: f32x3 = cross(e2, f3); if project6(ef23, p1, p2, p3, q1, q2, q3) == 0 { return false; }
31   let e3: f32x3 = p1 - p3; let ef31: f32x3 = cross(e3, f1); if project6(ef31, p1, p2, p3, q1, q2, q3) == 0 { return false; }
32   let ef32: f32x3 = cross(e3, f2); if project6(ef32, p1, p2, p3, q1, q2, q3) == 0 { return false; }
33   let ef33: f32x3 = cross(e3, f3); if project6(ef33, p1, p2, p3, q1, q2, q3) == 0 { return false; }
34   let g1: f32x3 = cross(e1, n1); if project6(g1, p1, p2, p3, q1, q2, q3) == 0 { return false; }
35   let g2: f32x3 = cross(e2, n1); if project6(g2, p1, p2, p3, q1, q2, q3) == 0 { return false; }
36   let g3: f32x3 = cross(e3, n1); if project6(g3, p1, p2, p3, q1, q2, q3) == 0 { return false; }
37   let h1: f32x3 = cross(f1, m1); if project6(h1, p1, p2, p3, q1, q2, q3) == 0 { return false; }
38   let h2: f32x3 = cross(f2, m1); if project6(h2, p1, p2, p3, q1, q2, q3) == 0 { return false; }
39   let h3: f32x3 = cross(f3, m1); if project6(h3, p1, p2, p3, q1, q2, q3) == 0 { return false; }
40   return true;
41 }
42 func intersects(a: Triangle, b: Triangle) -> bool {
43   return SAT_triangle_intersection(a.p0, a.p1, a.p2, b.p0, b.p1, b.p2);
44 }
45 func intersects(a: AABB, b: AABB) -> bool {
46   let low: f32x3 = max(a.low, b.low); let high: f32x3 = min(a.high, b.high); return all(low <= high);
47 }
48
49 func collision_detection(bvh1: BVH, bvh2: BVH, r: mut set[(Triangle, Triangle)]) =
50   match bvh1 {
51   | Interior(low1, high1, left1, right1) ->
52     match bvh2 {
53     | Interior(low2, high2, left2, right2) ->
54       if intersects(AABB(low1, high1), AABB(low2, high2)) {
55         collision_detection(left1, left2, r); collision_detection(left1, right2, r);
56         collision_detection(right1, left2, r); collision_detection(right1, right2, r);
57       }
58     | Leaf(low2, high2, _, _) ->
59       if intersects(AABB(low1, high1), AABB(low2, high2)) {
60         collision_detection(left1, bvh2, r); collision_detection(right1, bvh2, r);
61       }
62     }
63   | Leaf(low1, high1, _, data1) ->
64     match bvh2 {
65     | Interior(_, _, left2, right2) ->
66       if intersects(AABB(low1, high1), AABB(low2, high2)) {
67         collision_detection(bvh1, left2, r); collision_detection(bvh1, right2, r);
68       }
69     | Leaf(low2, high2, _, data2) ->
70       if intersects(AABB(low1, high1), AABB(low2, high2)) {
71         foreach t1 in data1 {
72           foreach t2 in data2 {
73             if intersects(t1, t2) { r.insert((t1, t2)); }
74           }
75         }
76       }
77     }
78   }

```

Received 2025-11-07; accepted 2026-04-03